

SELF-STUDY // ZERO TO MOBILE RE

COMPLETE HANDBOOK • 8 PHASES

# Reverse Engineering Mastery

Ek hi jagah — saare 8 phases ki poori details. Detailed Hinglish notes, code ki line-by-line explanation, aur har phase ke interview Q&A (Hinglish + English).

re@learning: ~/complete-handbook

```
$ cat roadmap.md | phases 0..7
# Foundation → Assembly → Tools → Static → Dynamic → Practice → Specialization → Mobile
$ status: ALL PHASES LOADED ✓
```

8 Phases

Code + Line-by-line

Interview Q&A ×2

IN Hinglish

Reverse Engineering Mastery • Complete Handbook • Phase 0 → 7

# Contents — Kya-kya hai is Handbook mein

---

## P0 Foundation

Neev — number systems, memory, C basics aur compilation

## P1 Assembly Language

Machine ki bhasha — registers, instructions, stack, function calls

## P2 Tools

Ghidra, x64dbg aur helper tools — .exe ki ghadi kholne ke aujaar

## P3 Static Analysis

Bina chalaaye code padh ke samajhna — strings, functions, decompiler

## P4 Dynamic Analysis

Chalate hue samajhna — breakpoints, live values, patching

## P5 Real Practice

Asli maidan — crackmes, CTF, malware analysis aur VM lab

## P6 Specialization

Apna raasta chuno — malware, exploits, mobile, crypto, firmware

## P7 Mobile RE

Android (APK, smali) aur iOS (IPA) apps reverse karna

**Har phase mein:** Detailed Notes (Hinglish, examples + code line-by-line) • Interview Q&A (Hinglish) • Interview Q&A (English). Phase 0 se shuru karo, ek-ek karke aage badho.

## PHASE 0

# Foundation

Neev – number systems, memory, C basics aur compilation

## Phase 0 — Foundation (Neev Majboot Karo)

**Goal:** Reverse Engineering seekhne se pehle computer andar se kaise kaam karta hai — ye samajhna. Ye neev hai. Agar ye kachhi rahi, toh aage sab kuch upar se nikal jaayega. Time: **2-3 hafte**.

### 0.0 — Sabse Pehle: Ye Phase Kyun?

Socho tum ek doctor banna chahte ho. Kya seedha operation seekhoge? Nahi. Pehle body ke parts (anatomy) padhoge. Waise hi RE mein, pehle "computer ki anatomy" samajhni padegi — numbers kaise store hote hain, memory kaise kaam karti hai, program kaise banta hai.

Is phase mein 4 cheezein seekhenge: 1. **Number Systems** (Binary, Hex, Decimal) 2. **Memory & Data** (RAM, bits, bytes, endianness) 3. **C Programming basics** (kyunki zyada tar programs C mein bane hote hain) 4. **Compilation** (source code se `.exe` kaise banta hai)

### 0.1 — Number Systems (Binary, Hex, Decimal)

#### Computer sirf 0 aur 1 samajhta hai

Humari duniya **decimal** (0-9, 10 digits) use karti hai. Lekin computer ke andar sirf electricity hai — ya toh current hai (1) ya nahi hai (0). Isi ko **Binary** kehte hain.

- **Binary (Base 2):** sirf `0` aur `1`
- **Decimal (Base 10):** `0` se `9` (hum log)
- **Hexadecimal / Hex (Base 16):** `0-9` phir `A,B,C,D,E,F`

## Hex kyun? (RE ka favourite)

Binary bahut lambi hoti hai. `11111111` likhna mushkil. Isliye RE mein hum **Hex** use karte hain — chhoti aur aasan. Har hex digit = 4 binary digits.

```
Binary:  1111 1111
Hex:     F   F   = 0xFF
```

`0x` prefix ka matlab hai "ye hex number hai". Jaise `0x41` , `0xFF` . RE tools mein sab jagah `0x` dikhega.

## Conversion Table (yaad rakhne layak)

| Decimal | Binary   | Hex  |
|---------|----------|------|
| 0       | 0000     | 0x0  |
| 5       | 0101     | 0x5  |
| 10      | 1010     | 0xA  |
| 15      | 1111     | 0xF  |
| 16      | 10000    | 0x10 |
| 65      | 1000001  | 0x41 |
| 255     | 11111111 | 0xFF |

## Real Example — Letter 'A'

Computer mein har letter ek number hota hai (ASCII code):

```
Letter 'A' = Decimal 65 = Binary 01000001 = Hex 0x41
Letter 'B' = Decimal 66 = Hex 0x42
Letter 'a' = Decimal 97 = Hex 0x61
```

RE karte waqt jab tumhe memory mein `41 42 43` dikhega, tum turant samajh jaoge ki wo `"ABC"` hai. Ye skill baar-baar kaam aayegi!

## Practice

- Decimal 200 ko hex mein convert karo (answer: `0xC8`)
- Hex `0x2A` ko decimal mein (answer: `42`)
- Windows calculator ko "Programmer mode" mein rakh ke practice karo — ye instant convert karta hai.

## 0.2 — Memory, Bits & Bytes

### Bits aur Bytes

- **Bit** = ek `0` ya `1` (sabse chhoti unit)
- **Byte** = 8 bits (`11111111`) — ek byte se 0 se 255 tak number ban sakta hai
- Aage: 1 KB = 1024 bytes, 1 MB = 1024 KB...

```
1 byte = 8 bits = 2 hex digits
Example: 0xFF = 11111111 = 255 (ek byte ki max value)
```

### Memory (RAM) — Ghar ke address jaisa

RAM ko ek badi building socho jismein crore-o flats hain. Har flat ka ek **address** hai. Computer har data ko kisi address pe rakhta hai.

| Address  | Value (byte) |
|----------|--------------|
| 0x1000 → | 0x41 ('A')   |
| 0x1001 → | 0x42 ('B')   |
| 0x1002 → | 0x43 ('C')   |

RE mein hum baar-baar "is address pe kya value hai?" ye dekhte hain. Debugger memory ko exactly aise hi dikhata hai.

### Endianness (★ Ye confuse karta hai — dhyan se)

Jab ek number ek byte se bada ho (jaise 4-byte number), toh usko memory mein kis order mein rakhein? Do tarike hain:

- **Little Endian:** chhota (least significant) byte pehle. **Intel/AMD (x86/x64) yahi use karte hain.**
- **Big Endian:** bada byte pehle (network protocols mein common).

**Example:** Number `0x12345678` ko memory mein store karna:

```
Little Endian (Intel PC):  78 56 34 12  <- ULTA! bytes reversed
Big Endian:                12 34 56 78  <- seedha
```

Beginners yahin confuse hote hain. Jab debugger mein `78 56 34 12` dikhe, actual number `0x12345678` hai. Bas byte order ulta hai. Ye yaad rakho.

## 0.3 — C Programming Basics (RE Ki Jaan)

Zyada tar software (Windows, games, malware) **C/C++** mein bane hote hain. Agar tum C samajhte ho, toh decompiled code (Ghidra ka output) padhna aasan ho jaayega — kyunki wo C jaisa hi dikhta hai.

### Ek simple C program

```
#include <stdio.h>           // input/output ke functions laane ke liye (printf)

int main() {                 // main = program yahi se shuru hota hai
    int age = 25;            // 'age' naam ka box banaya, usme 25 rakha
    printf("Age: %d", age);  // screen pe "Age: 25" chhaapo (%d = number)
    return 0;               // program theek se khatam (0 = success)
}
```

**Line by line samjho:** - `#include <stdio.h>` — ye "library" laata hai jisme `printf` jaisa function hota hai. Isse pehle likhna padta hai. - `int main()` — har C program `main` se chalu hota hai. `int` matlab ye ek number return karega. - `int age = 25;` — `int` = integer (poora number). `age` variable ka naam. `= 25` uski value. - `printf("Age: %d", age);` — screen pe print. `%d` ki jagah `age` ki value (25) aa jaayegi. - `return 0;` — program ko batana ki sab theek raha (0 = no error).

## Zaroori Concepts

### 1. Variables & Data Types

```
int    count = 10;          // poora number (integer)
char   letter = 'A';        // ek character (asal mein number 65)
float  price = 9.99;        // decimal number
```

## 2. Pointers (★ ★ Sabse important, RE ka dil)

Pointer ek variable hai jo kisi doosre variable ka **address** rakhta hai (value nahi, ghar ka pata).

```
int x = 5; // x mein 5 hai (maano address 0x1000 pe)
int *p = &x; // p mein x ka ADDRESS hai (0x1000), value nahi
printf("%d", *p); // *p matlab "us address pe jo value hai" = 5
```

**Line by line:** - `int x = 5;` — normal variable, value 5. - `int *p = &x;` — `*` batata hai ye pointer hai. `&x` matlab "x ka address". Toh `p` ke andar x ka address hai. - `*p` — `*` yahan "dereference" hai = "is address pe jaake value nikaalo" = 5.

Pointers samajh gaye toh RE ka 40% clear. Kyunki assembly mein sab kuch addresses se hota hai.

`&` = address lena, `*` = us address pe jaana.

## 3. Functions, Loops, If-else

```
int add(int a, int b) { // 'add' function, do numbers leta hai
    return a + b; // dono ko jodke wapas bhejta hai
}

for (int i = 0; i < 5; i++) { // loop: i=0 se 4 tak, 5 baar chalega
    printf("%d ", i); // print: 0 1 2 3 4
}

if (age >= 18) { // condition check
    printf("Adult");
} else {
    printf("Minor");
}
```

## 4. Arrays & Strings

```
int nums[3] = {10, 20, 30}; // 3 numbers ka array
char name[] = "Amit"; // string = characters ka array (A,m,i,t,\0)
```

String ke end mein ek chhupa `\0` (null byte) hota hai jo batata hai "string yahan khatam". RE mein ye dikhega.

## Practice

- Ek `hello.c` likho, compile karo, chalao.
- Pointer wala program likho aur samjho `&` aur `*` kaise kaam karte hain.

## 0.4 — Compilation: Source Code Se .exe Tak

Ye samajhna **bahut zaroori** hai, kyunki RE isi process ka **ulta** hai.

### Forward Process (normal programming)

```
hello.c      →  Compiler  →  hello.exe
(hum likhte hain)    (gcc/msvc)    (computer chalata hai)
```

Detail mein:

Source Code (.c)

↓ (Preprocessor — #include waale kaam)

Expanded Code

↓ (Compiler)

Assembly Code (.asm)

↓ (Assembler)

Machine Code / Object (.obj)

↓ (Linker — libraries jodta hai)

Executable (.exe)

### Reverse Process (RE — hum ye karte hain)

```
hello.exe  →  Disassembler  →  Assembly  →  (hum samajhte hain logic)
              (Ghidra/IDA)              →  Decompiler → C-jaisa code
```

Yaad rakho: compile karte waqt bahut si information kho jaati hai — variable ke naam ( `age` , `price` ), comments, sab gayab. Isliye RE mushkil hai — hume ye khoyi hui info wapas guess karni padti hai.

### Khud Karke Dekho (Best Practice)

`gcc -S` command source ko assembly mein badalti hai (compile nahi karti, sirf assembly dikhati hai):



```
gcc -S hello.c -o hello.s      # hello.s mein assembly milegi
gcc hello.c -o hello.exe      # normal compile
```

**Sabse best tool: [godbolt.org](https://godbolt.org)** - Left side mein C likho, right side mein turant assembly dikhegi. - Ye Phase 1 (Assembly) ke liye bhi bahut kaam aayega. Abhi se use karna shuru karo.



## Phase 0 — Hands-On Checklist

- [ ] Windows Calculator "Programmer mode" mein hex ↔ decimal convert kiya
- [ ] `'A'` = 65 = 0x41 samajh liya (ASCII)
- [ ] Little vs Big endian ka farak samajh liya ( `0x12345678` → `78 56 34 12` )
- [ ] Ek `hello.c` likha, compile kiya, chalaya
- [ ] Pointer wala program likha ( `&x` aur `*p` )
- [ ] godbolt.org pe C likh ke assembly dekhi



## Key Takeaways (Ek Line Mein)

1. Computer sirf **binary** (0/1) samajhta hai; hum **hex** (0x..) use karte hain kyunki chhoti hoti hai.
2. **1 byte = 8 bits = 2 hex digits**; ek byte 0-255 tak.
3. Har data RAM mein ek **address** pe hota hai.
4. **Little endian** = bytes ulte store hote hain (Intel PC). `0x12345678` → `78 56 34 12` .
5. **Pointer** = address rakhne wala variable. `&` = address lo, `*` = address pe jaao. (RE ka dil)
6. Compile = source → exe (info kho jaati hai). RE = exe → samajhna (info wapas guess karna).

✅ **Phase 0 Complete Jab:** Tum hex-decimal convert kar sako, ek chhota C program (pointer ke saath) likh-compile kar sako, aur memory/endianness ka clear idea ho. Ab Phase 1 (Assembly) ke liye ready ho! 🚀

# Phase 0 — Interview Questions & Answers (Hinglish)

---

Ye important theory questions hain jo interview mein Phase 0 (Foundation) se puche ja sakte hain. Har jawaab simple Hinglish mein — jaise tum interviewer ko bologe.

## Q1. Reverse Engineering kya hai?

**Ans:** Reverse Engineering ka matlab hai kisi cheez ko "ulta" samajhna — jab humein source code nahi pata, tab hum ek program (jaise `.exe`) ko todkar, andar dekhkar samajhte hain ki wo kaise kaam karta hai. Jaise ek band ghadi ko kholkar samajhna ki gears kaise chalte hain, waise hi software pe.

## Q2. Computer sirf binary (0 aur 1) kyun samajhta hai?

**Ans:** Computer ke andar electronic circuits hote hain jinme sirf do states hoti hain — current on (1) ya off (0). Isiliye computer sab kuch binary mein store aur process karta hai. Hardware level pe sirf yahi do cheezein possible hain.

## Q3. Hexadecimal kyun use karte hain, binary kyun nahi?

**Ans:** Binary bahut lambi hoti hai — jaise `11111111`. Hex chhoti aur padhne mein aasan hai — wahi cheez `0xFF` ho jaati hai. Ek hex digit exactly 4 binary digits (bits) ko represent karta hai, isliye conversion bhi seedha hai. RE tools memory aur values hex mein hi dikhate hain.

## Q4. 1 byte mein kitni values aa sakti hain?

**Ans:** 1 byte = 8 bits. 8 bits se  $2^8 = 256$  alag values ban sakti hain, matlab 0 se 255 tak. Hex mein ye `0x00` se `0xFF` tak hota hai.

## Q5. Memory address kya hota hai?

**Ans:** RAM ek badi building jaisi hai jismein har "flat" ka ek unique number hota hai — usi ko memory address kehte hain. Computer har data (variable, code) ko kisi address pe rakhta hai, taaki baad mein use dhoond sake. RE mein hum baar-baar dekhte hain ki kisi address pe kya value hai.

## Q6. Endianness kya hoti hai? Little aur Big endian mein kya farak hai?

**Ans:** Endianness batati hai ki ek multi-byte number memory mein kis order mein store hoga. - **Little Endian:** sabse chhota (least significant) byte pehle store hota hai. Intel/AMD (x86/x64) yahi use karte hain. - **Big Endian:** sabse bada byte pehle. Network protocols mein common.

Example: `0x12345678` little endian mein memory mein `78 56 34 12` (ulta) dikhega, big endian mein `12 34 56 78`.

## Q7. Pointer kya hota hai? Isse RE mein kya importance hai?

**Ans:** Pointer ek variable hai jo kisi doosre variable ki **value nahi, balki address** rakhta hai. `&x` se x ka address milta hai, aur `*p` se us address pe rakhi value milti hai. RE mein pointer bahut important hai kyunki assembly level pe zyada tar kaam addresses ke through hota hai — data ko address se access kiya jaata hai.

## Q8. ASCII kya hai? 'A' ki value kya hoti hai?

**Ans:** ASCII ek standard hai jo har character ko ek number deta hai, taaki computer text ko store kar sake. Jaise `'A'` = 65 (hex `0x41`), `'a'` = 97 (`0x61`), `'0'` = 48. RE mein jab memory mein `41 42 43` dikhta hai, wo actually `"ABC"` hota hai.

## Q9. Compilation process kya hai? Source code se .exe kaise banta hai?

**Ans:** Steps hote hain: 1. **Preprocessor** — `#include` jaise kaam handle karta hai. 2. **Compiler** — C code ko assembly mein badalta hai. 3. **Assembler** — assembly ko machine code (object file) mein. 4. **Linker** — libraries jodkar final `.exe` banata hai.

RE isi ka **ulta** hai — hum `.exe` se wapas samajhne ki koshish karte hain.

## Q10. Compile karne pe kaunsi information kho jaati hai?

**Ans:** Compile hone pe variable ke naam (jaise `age`, `price`), function ke asli naam, comments, aur data types ki clear info kho jaati hai. Machine code mein sirf raw instructions bachti hain. Isliye RE mushkil hota hai — hume ye khoyi hui information wapas guess karni padti hai.

## Q11. Machine code aur Assembly mein kya farak hai?

**Ans: Machine code** pure numbers (0s aur 1s / bytes) hote hain jo CPU seedha chalata hai. **Assembly** usi machine code ka human-readable version hai — jaise `mov eax, 5`. Har assembly instruction ek machine code se map hoti hai. Assembly humein padhne mein aasan hai.

---

### Q12. Stack aur Heap mein basic farak kya hai?

**Ans:** Dono memory ke hisse hain jahan program data rakhta hai. - **Stack:** automatic, fast, function ke local variables aur call info yahan aati hai. Order fixed hota hai (LIFO — Last In First Out). - **Heap:** manual/dynamic memory, jise program chalte-chalte maangta hai (jaise C mein `malloc`). Zyada flexible but manage karna padta hai.

---

### Q13. Bit aur Byte mein farak?

**Ans: Bit** sabse chhoti unit hai — ek `0` ya `1`. **Byte** = 8 bits ka group. Ek byte se 0-255 tak number ban sakta hai. Data usually bytes mein measure hota hai (KB, MB, GB).

---

### Q14. RE seekhne ke liye C language kyun zaroori hai?

**Ans:** Kyunki zyada tar software C/C++ mein bane hote hain, aur decompiler (jaise Ghidra) ka output C jaisa dikhta hai. Agar tum C samajhte ho — especially pointers, arrays, functions — toh decompiled code padhna bahut aasan ho jaata hai.

# Phase 0 — Interview Questions & Answers (English)

---

The same important theory questions from Phase 0 (Foundation), answered in clear English. Use this to phrase your exact answer in a formal interview.

## Q1. What is Reverse Engineering?

**Ans:** Reverse Engineering is the process of understanding how something works by taking it apart — without having its source code. In software, we take a compiled program (like an `.exe`) and analyze it to figure out its internal logic. It's like opening a closed watch to understand how the gears drive it, but applied to software.

## Q2. Why does a computer only understand binary (0s and 1s)?

**Ans:** A computer's electronic circuits have only two stable states — current on (**1**) or off (**0**). Because hardware can reliably represent only these two states, everything is stored and processed in binary.

## Q3. Why do we use hexadecimal instead of binary?

**Ans:** Binary is long and hard to read (e.g., `11111111`). Hexadecimal is compact and readable — the same value becomes `0xFF`. Each hex digit maps exactly to 4 binary bits, so conversion is straightforward. RE tools display memory and values in hex.

## Q4. How many values can a single byte hold?

**Ans:** One byte = 8 bits. With 8 bits you get  $2^8 = 256$  distinct values, i.e. **0 to 255** (hex `0x00` to `0xFF`).

## Q5. What is a memory address?

**Ans:** RAM is like a large building where every "flat" has a unique number — that number is the memory address. The computer stores every piece of data (variables, code) at some address so it can locate it later. In RE we frequently inspect what value sits at a given address.

## Q6. What is endianness? Difference between little and big endian?

**Ans:** Endianness defines the byte order in which a multi-byte number is stored in memory. - **Little Endian:** the least significant byte is stored first. Used by Intel/AMD (x86/x64). - **Big Endian:** the most significant byte is stored first. Common in network protocols.

Example: `0x12345678` is stored as `78 56 34 12` in little endian, and `12 34 56 78` in big endian.

---

## Q7. What is a pointer? Why is it important in RE?

**Ans:** A pointer is a variable that holds the **address** of another variable rather than its value. `&x` gives the address of `x`, and `*p` gives the value stored at that address. Pointers are crucial in RE because at the assembly level most data access happens through addresses.

---

## Q8. What is ASCII? What is the value of 'A'?

**Ans:** ASCII is a standard that assigns a number to each character so computers can store text. For example, `'A'` = 65 (hex `0x41`), `'a'` = 97 (`0x61`). In RE, bytes like `41 42 43` in memory actually represent the string `"ABC"`.

---

## Q9. What is the compilation process — how does source code become an .exe?

**Ans:** The steps are: 1. **Preprocessor** — handles directives like `#include`. 2. **Compiler** — translates C code into assembly. 3. **Assembler** — converts assembly into machine code (object files). 4. **Linker** — links libraries and produces the final `.exe`.

Reverse Engineering is the reverse of this — going from `.exe` back to understanding the logic.

---

## Q10. What information is lost during compilation?

**Ans:** Variable names (like `age`, `price`), original function names, comments, and clear type information are lost. Only raw instructions remain in the machine code. This is why RE is hard — we must reconstruct that lost information by analysis.

---

## Q11. What is the difference between machine code and assembly?

**Ans:** **Machine code** is pure numbers (bytes) that the CPU executes directly. **Assembly** is a human-readable representation of that machine code, e.g., `mov eax, 5`. Each assembly instruction maps to machine code, but assembly is far easier for humans to read.

---

### Q12. What is the basic difference between the stack and the heap?

**Ans:** Both are regions of memory a program uses. - **Stack:** automatic and fast; stores local variables and function call information in LIFO (Last In, First Out) order. - **Heap:** dynamic memory requested at runtime (e.g., `malloc` in C). More flexible but must be managed manually.

---

### Q13. What is the difference between a bit and a byte?

**Ans:** A **bit** is the smallest unit — a single `0` or `1`. A **byte** is a group of 8 bits and can represent values from 0 to 255. Data sizes are usually measured in bytes (KB, MB, GB).

---

### Q14. Why is knowing C important for learning RE?

**Ans:** Most software is written in C/C++, and decompiler output (e.g., from Ghidra) resembles C. If you understand C — especially pointers, arrays, and functions — reading decompiled code becomes much easier.

## PHASE 1

# Assembly Language

Machine ki bhasha — registers, instructions, stack, function calls

## Phase 1 — Assembly Language (Machine Ki Bhasha)

**Goal:** Assembly padhna aur samajhna seekhna. Jab bhi tum `.exe` khologe, yahi dikhegi. Ye **sabse important phase** hai — isko achhe se karo. Time: **3-4 hafte**. Jaldbaazi bilkul nahi.

### 1.0 — Assembly Kyun Itni Important?

Jab ek program compile hota hai, wo machine code ban jaata hai. Ghidra/IDA jaise tools us machine code ko **Assembly** mein dikhate hain. Matlab jo bhi program tum reverse karoge, uski asli bhasha Assembly hi hogi.

Ek line yaad rakho: **"RE aati hai jab Assembly aati hai."** Baaki sab tools aur tricks bekaar hain agar tum assembly nahi padh sakte.

Ghabrao mat — assembly padhna seekhna hai, likhna nahi (abhi). Aur ye utni mushkil nahi jitni lagti hai, kyunki har instruction bahut chhota kaam karti hai.

### 1.1 — Assembly Hai Kya?

Assembly, machine code ka human-readable version hai. CPU ko diye jaane wale chhote-chhote commands (instructions).

**Example:**



```
mov eax, 5      ; eax naam ke dabbe mein 5 daalo
add eax, 3      ; eax mein 3 aur jodo (ab eax = 8)
```

- C ki ek line = kai assembly lines. Assembly bahut "low-level" hai — matlab CPU ke bahut kareeb.
- Har line ek chhota specific kaam karti hai: value move karo, jodo, compare karo, jump karo.

## Syntax ke do styles (dhyan rakho)

- **Intel syntax:** `mov eax, 5` (destination pehle) — Windows/IDA/Ghidra mein common. **Hum yahi use karenge.**
- **AT&T syntax:** `mov $5, %eax` (source pehle, `%` aur `$` ke saath) — Linux/GDB default.

Confusion se bachne ke liye ek hi style pe focus karo. Intel syntax RE mein zyada common hai.

## 1.2 — CPU Registers (Chhote Dabbe)

Registers = CPU ke andar bahut chhote, bahut fast storage boxes jahan calculation hoti hai. CPU pehle data ko register mein laata hai, phir kaam karta hai.

### General Purpose Registers (32-bit / 64-bit)

| 32-bit           | 64-bit           | Aam kaam                                             |
|------------------|------------------|------------------------------------------------------|
| <code>eax</code> | <code>rax</code> | Calculation, function ka return value yahan aata hai |
| <code>ebx</code> | <code>rbx</code> | General data                                         |
| <code>ecx</code> | <code>rcx</code> | Loop counter (counting)                              |
| <code>edx</code> | <code>rdx</code> | Data, multiplication/division mein help              |
| <code>esi</code> | <code>rsi</code> | Source (copy karte waqt "kahan se")                  |
| <code>edi</code> | <code>rdi</code> | Destination (copy karte waqt "kahan")                |

## Special Registers

| Register                            | Kaam                                                                                   |
|-------------------------------------|----------------------------------------------------------------------------------------|
| <code>esp</code> / <code>rsp</code> | <b>Stack Pointer</b> — stack ke top (upar) ko point karta hai                          |
| <code>ebp</code> / <code>rbp</code> | <b>Base Pointer</b> — current function ke stack frame ka base                          |
| <code>eip</code> / <code>rip</code> | <b>Instruction Pointer</b> — abhi kaunsi instruction chal rahi hai (★ bahut important) |

**EIP/RIP:** Ye batata hai CPU abhi kahan hai. Agar hum ise control kar lein (exploit mein), toh program hijack ho jaata hai. Isliye ye register RE aur hacking dono mein star hai.

## Register ke chhote hisse

Ek 64-bit register ke andar chhote versions bhi hote hain:

```
RAX (64-bit) — poora 8 bytes
└─ EAX (32-bit) — niche ke 4 bytes
    └─ AX (16-bit) — niche ke 2 bytes
        └─ AH | AL (8-bit) — AL sabse niche ka 1 byte
```

Jab code mein `al` dikhe, wo `eax` ka sabse chhota (1 byte) hissa hai. Ye choti values (jaise ek character) ke liye use hota hai.

## 1.3 — Common Instructions (Ratne nahi, samajhne hain)

Ye lists dekh ke ghabrana nahi — practice se sab yaad ho jaayega.

### Data Move

```
mov eax, 5      ; eax = 5 (value copy)
mov ebx, eax     ; ebx = eax (ek register se doosre mein)
mov [0x1000], eax ; memory address 0x1000 pe eax ki value likho
mov eax, [0x1000] ; address 0x1000 se value eax mein laao
lea eax, [ebx+4] ; eax = ebx+4 ka ADDRESS (value nahi) — "load effective address"
```

[ ] brackets ka matlab: "is address pe jaake dekho/likho" (pointer jaisa). Bina bracket = seedha value ya register.

## Arithmetic (ganit)

```
add eax, 3      ; eax = eax + 3
sub eax, 1      ; eax = eax - 1
inc eax         ; eax = eax + 1 (increment)
dec eax         ; eax = eax - 1 (decrement)
imul eax, 2     ; eax = eax * 2 (multiply)
xor eax, eax    ; eax = 0 (khud ke saath XOR = 0, ye common trick hai!)
```

`xor eax, eax` register ko 0 karne ka fast tarika hai. RE mein baar-baar dikhega — ise "eax = 0" samajhna.

## Compare & Jump (★ decisions yahan hote hain)

```
cmp eax, 10     ; eax aur 10 compare karo (result flags mein)
je label        ; Jump if Equal (agar barabar the)
jne label       ; Jump if Not Equal
jg label        ; Jump if Greater
jl label        ; Jump if Less
jmp label       ; hamesha jump (bina condition)
```

**Ye if-else banate hain!** `cmp` do cheezein compare karta hai, phir `je` / `jne` /etc. decide karte hain kahan jaana hai.

## Function Calls

```
call func       ; 'func' function ko bulao (aur wapas aane ka address yaad rakho)
ret             ; function se wapas jao (return)
push eax        ; eax ki value stack pe daalo (upar)
pop eax         ; stack se upar wali value eax mein nikaalo
```

# Cheat Sheet Table

| Instruction | Kaam                   | Yaad rakhne ka tarika |
|-------------|------------------------|-----------------------|
| mov         | copy data              | "move"                |
| add / sub   | jodna/ghatana          | + / -                 |
| cmp         | compare                | do cheez check        |
| jmp         | jump (always)          | seedha kudo           |
| je / jne    | jump if equal/not      | condition pe kudo     |
| call        | function bulao         | function call         |
| ret         | wapas                  | return                |
| push / pop  | stack pe daalo/nikaalo | stack                 |
| xor r,r     | register = 0           | zero karne ka trick   |
| lea         | address nikaalo        | pointer math          |

## 1.4 — Ek Real Example: C ↔ Assembly

Chalo ek chhota C program lete hain aur uski assembly samajhte hain.

### C Code

```
int main() {
    int a = 5;
    int b = 3;
    int c = a + b;    // c = 8
    return c;
}
```

## Assembly (approx, Intel syntax)

```
main:
    push ebp          ; purana base pointer bachao (stack pe)
    mov  ebp, esp      ; naya stack frame set karo
    mov  dword [ebp-4], 5 ; a = 5 (stack pe rakha)
    mov  dword [ebp-8], 3 ; b = 3
    mov  eax, [ebp-4]   ; eax = a (5)
    add  eax, [ebp-8]   ; eax = eax + b = 5+3 = 8
    mov  [ebp-12], eax  ; c = eax = 8
    mov  eax, [ebp-12]  ; return value eax mein daalo (8)
    pop  ebp           ; base pointer wapas
    ret               ; return
```

**Line by line samjho:** - `push ebp` / `mov ebp, esp` — har function shuru mein ye karta hai ("prologue"). Purana frame bachata hai, naya banata hai. - `mov dword [ebp-4], 5` — variable `a` stack pe `ebp-4` address pe hai, usme 5 daala. (`dword` = 4 bytes ka int) - `mov eax, [ebp-4]` — `a` ki value `eax` mein laayi. - `add eax, [ebp-8]` — usme `b` jod diya. Ab `eax = 8`. - `mov eax, [ebp-12]` — return karne ke liye value `eax` mein rakhi (return value hamesha `eax/rax` mein). - `pop ebp` / `ret` — frame wapas, function khatam ("epilogue").

Dekha? C ki 4 lines = assembly ki ~10 lines. Har local variable stack pe `[ebp-kuch]` pe rehta hai. Ye pattern har jagah dikhega — ise pehchano.

## If-else assembly mein kaisa dikhta hai

```
if (x == 10) { y = 1; }
else          { y = 2; }
```

```
cmp dword [ebp-4], 10 ; x ko 10 se compare
jne else_part        ; agar equal NAHI, else pe jao
mov dword [ebp-8], 1  ; y = 1 (if wala part)
jmp end
else_part:
mov dword [ebp-8], 2  ; y = 2 (else wala part)
end:
```

Pattern: `cmp` + conditional jump = if-else. Ye pehchan gaye toh RE bahut aasan.

## 1.5 — Stack Ko Deeply Samjho (★ ★)

Stack samajhna RE ka bada milestone hai. Stack ek "plate ka dher" jaisa hai — LIFO (Last In, First Out). Jo aakhri mein rakha, wahi pehle nikalta hai.

### Stack kya rakhta hai?

- Function ke **local variables**
- Function ke **arguments** (parameters)
- **Return address** (function khatam hone pe kahan wapas jaana hai)
- Purana **base pointer** (ebp)

### Stack Frame ka structure (ek function ka)

Higher address (upar)

|                    |                              |
|--------------------|------------------------------|
| function arguments |                              |
| return address     | <- call ke waqt push hua     |
| old ebp (saved)    | <- ebp yahan point karta hai |
| local variable a   | [ebp-4]                      |
| local variable b   | [ebp-8]                      |

Lower address (esp yahan, stack niche badhta hai)

**Important:** Stack **niche ki taraf** badhta hai (higher se lower address). `push` karne pe esp chhota hota hai, `pop` pe bada. Ye ulta lagta hai but yaad rakho.

### call aur ret internally kya karte hain

- `call func` = pehle wapas aane ka address stack pe **push** karta hai, phir func pe jump.
- `ret` = stack se wo address **pop** karta hai aur wahan wapas jump.

Isiliye **buffer overflow** attacks kaam karte hain — agar hum stack pe return address ko overwrite kar dein, toh program kahin aur "wapas" jaayega jahan hum chahein. (Ye advanced topic hai, abhi

bas idea rakho.)

## 1.6 — x86 vs x64 (32-bit vs 64-bit)

|               | x86 (32-bit)             | x64 (64-bit)                                                            |
|---------------|--------------------------|-------------------------------------------------------------------------|
| Registers     | <code>eax, ebx...</code> | <code>rax, rbx...</code> (aur <code>r8</code> - <code>r15</code> extra) |
| Address size  | 4 bytes                  | 8 bytes                                                                 |
| Function args | zyada tar stack pe       | pehle registers mein ( <code>rcx, rdx, r8, r9</code> Windows)           |
| Seekhna       | aasan (pehle karo)       | modern (baad mein)                                                      |

**Advice:** Pehle x86 (32-bit) achhe se samajho — simple hai. Phir x64 pe jaao. Concepts same hain, bas thoda badla hua.

## 1.7 — Calling Conventions (Functions ko args kaise milte hain)

Jab ek function ko call karte hain, arguments kaise pass hote hain — iske rules ko "calling convention" kehte hain.

- **x86 cdecl:** arguments stack pe push hote hain (ulte order mein). Return value `eax` mein.
- **x64 Windows:** pehle 4 args registers mein — `rcx, rdx, r8, r9`. Baaki stack pe. Return `rax` mein.
- **x64 Linux (System V):** pehle 6 args — `rdi, rsi, rdx, rcx, r8, r9`.

RE mein jab `call` dikhe, uske pehle wale `mov rcx, ...` / `push ...` dekho — wahi function ke arguments hain. Return value hamesha `eax` / `rax` mein aati hai.



## Phase 1 — Hands-On Practice

1. **godbolt.org** kholo. Chhote C programs likho, unki assembly dekho. C aur assembly ko side-by-side compare karo.
2. Ye programs try karo: simple addition, if-else, ek loop, ek function call.
3. Har assembly line ko English/Hinglish mein "translate" karo — ye best exercise hai.

4. Register kis line pe kya value rakhega — dimaag mein simulate karo.

---

## Key Takeaways (Ek Line Mein)

1. Assembly = machine code ka human-readable version. **RE ki asli bhasha.**
  2. **Registers** = CPU ke chhote fast dabbe. `eax` = calculation/return, `esp` = stack top, `eip` = abhi kahan.
  3. `[ ]` brackets = memory access (pointer jaisa).
  4. `cmp` + conditional jump ( `je` / `jne` ) = **if-else**. Loops mein bhi yahi.
  5. **Stack niche badhta hai**, LIFO hai, aur function ke local vars + return address rakhta hai.
  6. `call` return address push karta hai, `ret` use pop karke wapas jaata hai.
  7. Return value hamesha `eax` / `rax` mein.
  8. Pehle **x86** (aasan), phir **x64**.
- 

✅ **Phase 1 Complete Jab:** Tum ek simple assembly code padh ke samajh sako ki wo kya kar raha hai — jaise loop, if-else, addition, function call. Ye aa gaya toh RE ka sabse bada hurdle paar. Ab tools seekhne ki baari (Phase 2)! 🚀



# Phase 1 — Interview Questions & Answers (Hinglish)

Assembly Language se related important interview questions, simple Hinglish jawaab ke saath.

## Q1. Assembly language kya hai?

**Ans:** Assembly, machine code ka human-readable version hai. CPU ko diye jaane wale chhote-chhote commands (instructions) jaise `mov eax, 5`. Har assembly instruction seedha ek machine code se map hoti hai. RE mein jab hum `.exe` kholte hain, tools use assembly mein dikhate hain.

## Q2. Register kya hote hain?

**Ans:** Registers CPU ke andar bahut chhote aur bahut fast storage boxes hain jahan actual calculation hoti hai. CPU pehle data ko RAM se register mein laata hai, phir uspe kaam karta hai. Examples: `eax`, `ebx`, `ecx`, `esp`, `eip`.

## Q3. EAX, ESP, aur EIP registers ka kya kaam hai?

**Ans:** - **EAX** — general calculations aur function ka return value yahan aata hai. - **ESP** (Stack Pointer) — stack ke top (upar) ko point karta hai. - **EIP** (Instruction Pointer) — batata hai CPU abhi kaunsi instruction chala raha hai. Ye bahut important hai — exploit mein isko control karna hi program hijack karna hota hai.

## Q4. Intel aur AT&T syntax mein kya farak hai?

**Ans:** Dono assembly likhne ke styles hain. - **Intel:** `mov eax, 5` — destination pehle, source baad mein. Windows/IDA/Ghidra mein common. - **AT&T:** `mov $5, %eax` — source pehle, `%` aur `$` symbols ke saath. Linux/GDB mein default.

RE mein Intel syntax zyada use hota hai.

## Q5. `mov eax, [0x1000]` aur `mov eax, 0x1000` mein kya farak hai?

**Ans:** Brackets `[ ]` ka matlab "memory access" hota hai. - `mov eax, 0x1000` — eax mein seedha value 0x1000 daal do. - `mov eax, [0x1000]` — address 0x1000 pe jo value rakhi hai, wo eax mein laao (pointer ki tarah).

---

### Q6. `cmp` aur conditional jumps if-else kaise banate hain?

**Ans:** `cmp` do values ko compare karta hai aur result "flags" mein rakhta hai. Uske baad conditional jump ( `je` = jump if equal, `jne` = jump if not equal, `jg` = greater, `jl` = less) decide karta hai kahan jaana hai. Ye combination hi assembly mein if-else aur loops banata hai.

---

### Q7. Stack kya hai? Wo kis direction mein badhta hai?

**Ans:** Stack memory ka ek hissa hai jo LIFO (Last In First Out) tarique se kaam karta hai — jaise platon ka dher. Ye function ke local variables, arguments, return address, aur saved base pointer rakhta hai. Important baat: stack **niche ki taraf** (higher se lower address) badhta hai. `push` karne pe ESP chhota hota hai, `pop` pe bada.

---

### Q8. `call` aur `ret` instructions internally kya karte hain?

**Ans:** `call func` pehle wapas aane ka address (return address) stack pe **push** karta hai, phir `func` pe jump karta hai. `ret` stack se wahi return address **pop** karke wapas us jagah jump kar jaata hai. Isliye function khatam hone pe program sahi jagah lautta hai.

---

### Q9. Stack frame kya hota hai?

**Ans:** Har function call ka apna ek memory area stack pe hota hai — usi ko stack frame kehte hain. Ismein us function ke arguments, return address, saved old EBP, aur local variables hote hain. EBP (base pointer) frame ke base ko point karta hai, aur local variables usually `[ebp-4]`, `[ebp-8]` jaise access hote hain.

---

### Q10. x86 aur x64 mein kya farak hai?

**Ans:** - **x86 (32-bit):** registers `eax, ebx...`, addresses 4 bytes, function arguments zyada tar stack pe pass hote hain. - **x64 (64-bit):** registers `rax, rbx...` plus extra `r8 - r15`, addresses 8 bytes, pehle kuch arguments registers mein pass hote hain (Windows: `rcx, rdx, r8, r9`).

Pehle x86 seekhna aasan hota hai, phir x64.

---

### Q11. Calling convention kya hoti hai?

**Ans:** Calling convention wo rules hain jo batate hain ki function ko arguments kaise diye jaayenge aur return value kahan milegi. Example: x86 cdecl mein arguments stack pe push hote hain aur return value `eax` mein aati hai; x64 Windows mein pehle 4 args `rcx, rdx, r8, r9` registers mein jaate hain.

---

### Q12. `xor eax, eax` kya karta hai aur kyun common hai?

**Ans:** Kisi register ko khud ke saath XOR karne pe wo **0** ho jaata hai. `xor eax, eax` matlab `eax = 0`. Compiler isko `mov eax, 0` ki jagah use karta hai kyunki ye thoda fast aur chhota hota hai. RE mein ye pattern bahut dikhega — ise "eax ko zero karna" samajhna.

---

### Q13. `lea` instruction kya karti hai?

**Ans:** `lea` (Load Effective Address) kisi address ko calculate karke register mein daal deti hai — value nahi, sirf address. Jaise `lea eax, [ebx+4]` eax mein `ebx+4` ka address rakhega. Compiler ise pointer math aur kabhi-kabhi fast arithmetic ke liye bhi use karta hai.

---

### Q14. Function ka return value kahan milta hai?

**Ans:** Convention ke hisaab se function ka return value hamesha accumulator register mein aata hai — x86 mein `eax`, x64 mein `rax`. RE mein jab `call` ke baad `eax` ki value use ho rahi ho, wo function ka result hai.

---

### Q15. Register ke chhote hisse (jaise AL, AX) kya hote hain?

**Ans:** Ek bade register ke andar chhote versions hote hain. Jaise `RAX` (64-bit) ke andar `EAX` (32-bit), uske andar `AX` (16-bit), aur uske andar `AH` / `AL` (8-bit) hote hain. `AL` sabse niche ka 1 byte hai — chhoti values (jaise ek character) ke liye use hota hai.

# Phase 1 — Interview Questions & Answers (English)

---

The same Assembly Language interview questions, answered in clear English for formal interviews.

## Q1. What is assembly language?

**Ans:** Assembly is the human-readable form of machine code — small instructions given to the CPU, such as `mov eax, 5`. Each assembly instruction maps directly to machine code. When we open an `.exe` in RE tools, they present it as assembly.

## Q2. What are registers?

**Ans:** Registers are very small, very fast storage locations inside the CPU where actual computation happens. The CPU loads data from RAM into a register, operates on it, and stores the result. Examples: `eax`, `ebx`, `ecx`, `esp`, `eip`.

## Q3. What is the role of the EAX, ESP, and EIP registers?

**Ans:** - **EAX** — used for general calculations and holds a function's return value. - **ESP** (Stack Pointer) — points to the top of the stack. - **EIP** (Instruction Pointer) — indicates the instruction the CPU is currently executing. It's crucial: controlling EIP is what lets an attacker hijack program flow.

## Q4. What is the difference between Intel and AT&T syntax?

**Ans:** They are two styles of writing assembly. - **Intel:** `mov eax, 5` — destination first, then source. Common on Windows/IDA/Ghidra. - **AT&T:** `mov $5, %eax` — source first, with `%` and `$` symbols. Default on Linux/GDB.

Intel syntax is more common in RE.

## Q5. What is the difference between `mov eax, [0x1000]` and `mov eax, 0x1000` ?

**Ans:** Brackets `[ ]` mean memory access. - `mov eax, 0x1000` — put the literal value 0x1000 into eax. - `mov eax, [0x1000]` — load the value stored at memory address 0x1000 into eax (like dereferencing a pointer).

---

## Q6. How do `cmp` and conditional jumps form if-else logic?

**Ans:** `cmp` compares two values and sets CPU flags based on the result. A following conditional jump (`je` = jump if equal, `jne` = jump if not equal, `jg` = greater, `j1` = less) then decides where to go. This combination implements if-else statements and loops in assembly.

---

## Q7. What is the stack and in which direction does it grow?

**Ans:** The stack is a region of memory that works in LIFO (Last In, First Out) order, like a stack of plates. It stores a function's local variables, arguments, return address, and saved base pointer. Importantly, the stack grows **downward** (from higher to lower addresses): `push` decreases ESP, `pop` increases it.

---

## Q8. What do the `call` and `ret` instructions do internally?

**Ans:** `call func` first **pushes** the return address onto the stack, then jumps to `func`. `ret` **pops** that return address off the stack and jumps back to it, so execution resumes right after the original call.

---

## Q9. What is a stack frame?

**Ans:** Each function call has its own region on the stack called a stack frame. It contains that function's arguments, the return address, the saved old EBP, and its local variables. EBP (base pointer) points to the frame's base, and locals are typically accessed as `[ebp-4]`, `[ebp-8]`, etc.

---

## Q10. What is the difference between x86 and x64?

**Ans:** - **x86 (32-bit):** registers `eax, ebx...`, 4-byte addresses, and function arguments mostly passed on the stack. - **x64 (64-bit):** registers `rax, rbx...` plus extra `r8 - r15`, 8-byte addresses, and the first few arguments passed in registers (Windows: `rcx, rdx, r8, r9`).

x86 is easier to learn first, then move to x64.

---

## Q11. What is a calling convention?

**Ans:** A calling convention is the set of rules defining how arguments are passed to a function and where the return value is placed. For example, x86 cdecl pushes arguments on the stack and returns the value in `eax`; x64 Windows passes the first four arguments in `rcx, rdx, r8, r9`.

---

### Q12. What does `xor eax, eax` do and why is it common?

**Ans:** XOR-ing a register with itself sets it to **0**, so `xor eax, eax` means `eax = 0`. Compilers use it instead of `mov eax, 0` because it's slightly faster and smaller in machine code. It's a very common pattern in RE — recognize it as "zeroing the register."

---

### Q13. What does the `lea` instruction do?

**Ans:** `lea` (Load Effective Address) computes an address and stores it in a register — the address, not the value at it. For example, `lea eax, [ebx+4]` puts the address `ebx+4` into `eax`. Compilers use it for pointer arithmetic and sometimes for fast arithmetic.

---

### Q14. Where is a function's return value stored?

**Ans:** By convention, the return value is placed in the accumulator register — `eax` on x86, `rax` on x64. In RE, if `eax` is used right after a `call`, that value is the function's result.

---

### Q15. What are the smaller sub-registers like AL and AX?

**Ans:** A large register contains smaller sub-registers. `RAX` (64-bit) contains `EAX` (32-bit), which contains `AX` (16-bit), which contains `AH` / `AL` (8-bit). `AL` is the lowest byte — used for small values like a single character.

## PHASE 2

# Tools

Ghidra, x64dbg aur helper tools – .exe ki ghadi kholne ke aujaar

## Phase 2 — Tools Se Dosti (Aujaar Seekho)

**Goal:** Wo tools seekhna jinse hum `.exe` ki "ghadi" kholte hain. Ab tak theory thi, ab practical tools. Time: **2-3 hafte**.

### 2.0 — Tools Kyun Zaroori?

Assembly padhna aa gaya (Phase 1), lekin `.exe` ko assembly mein badalta kaun hai? **Tools**. Ye tools raw machine code ko humein padhne layak (assembly / C-jaisa) code mein dikhate hain, program ko chalate-rokte hain, aur memory dikhate hain.

Tools 3 category ke hote hain: 1. **Disassembler / Decompiler** — `.exe` → assembly / C-jaisa code (static) 2. **Debugger** — program ko chalate hue rok-rok ke dekhna (dynamic) 3. **Helper tools** — file inspect karna, strings nikaalna, hex dekhna

### 2.1 — Disassembler vs Decompiler (Farak Samjho)

|          | Disassembler  | Decompiler                  |
|----------|---------------|-----------------------------|
| Output   | Assembly code | C-jaisa high-level code     |
| Padhna   | thoda mushkil | aasan                       |
| Accuracy | 100% sahi     | approximate (kabhi galat)   |
| Example  | IDA, objdump  | Ghidra decompiler, Hex-Rays |

- **Disassembler:** machine code → assembly. Ye exact hota hai.

- **Decompiler:** assembly → C-jaisa code. Ye padhna aasan hai, but variable naam ajeeb ( `local_8` , `uVar1` ) aur kabhi galti bhi karta hai.

**Best strategy:** Dono use karo. Decompiler se overall logic samjho, aur jahan doubt ho wahan assembly (disassembler) dekho.

## 2.2 — Main Tools (Comparison)

| Tool                  | Kya Hai                                                       | Cost | Beginner?       |
|-----------------------|---------------------------------------------------------------|------|-----------------|
| <b>Ghidra</b> ★       | NSA ka banaya, free, powerful. Disassembler + decompiler dono | Free | ✓ Best se shuru |
| <b>IDA Free</b>       | Industry standard, free version (64-bit disassembler)         | Free | ✓               |
| <b>x64dbg</b>         | Windows debugger (runtime analysis)                           | Free | ✓               |
| <b>Cutter / Rizin</b> | Open source, modern UI                                        | Free | ✓               |
| <b>Binary Ninja</b>   | Achha UI, powerful                                            | Paid | ⚠ baad mein     |
| <b>GDB</b>            | Linux ka classic debugger                                     | Free | Linux ke liye   |

**Recommendation:** **Ghidra + x64dbg** se shuru karo. Dono free hain. Ghidra static analysis ke liye, x64dbg dynamic ke liye. Yahi combo 90% kaam kar dega.

## 2.3 — Ghidra (Sabse Zaroori Tool)

Ghidra free hai aur **decompiler** bhi deta hai (assembly ko C-jaisa dikhata hai) — isiliye beginners ke liye best.

### Ghidra Setup

1. Java (JDK 17+) install karo (Ghidra ko chahiye).
2. Ghidra download karo (official NSA GitHub se).
3. Run karo, ek **Project** banao.
4. Apna `.exe` import karo (File → Import File).
5. Analysis chalao ("Analyze" — Yes bolo, defaults theek hain).

### Ghidra ke main windows (parts)

- **Listing** (beech mein) — assembly code, address ke saath.



- **Decompiler** (right side) — C-jaisa code. ★ Sabse zyada yahi padhoge.
- **Symbol Tree** (left) — functions, imports, exports ki list.
- **Program Trees / Data Type Manager** — structure aur types.

## Ghidra mein basic kaam

- **Function dhoondo:** Symbol Tree → Functions. `main` ya `entry` dhoondo.
- **Rename karo:** Kisi variable/function pe click → `L` dabao → naya naam do (jaise `local_8` ko `user_input`). Isse code padhna aasan hota hai.
- **Cross-references (XREF):** Kisi function pe right-click → "References" → dekho use kahan-kahan call hua. ★ Bahut useful.
- **Comments:** `;` dabao ke apne notes likho.

**Tip:** Ghidra mein rename aur comment karte raho jaise-jaise samajhte ho. Ye "map banane" jaisa hai — jitna map banega, utna aasan.

## 2.4 — x64dbg (Windows Debugger)

x64dbg program ko **chalate hue** rok-rok ke dekhne ke liye hai (dynamic analysis — Phase 4 mein detail).

### x64dbg ke main parts

- **CPU window** — assembly instructions (abhi kaunsi chal rahi).
- **Registers** (right) — live values (eax, ebx, eip...).
- **Dump** (niche) — memory ka hex view.
- **Stack** (right-niche) — stack ki live values.

### Basic controls

| Key             | Kaam                                            |
|-----------------|-------------------------------------------------|
| <code>F2</code> | Breakpoint lagao/hatao (yahan ruk jaao)         |
| <code>F7</code> | Step Into (function ke andar jao)               |
| <code>F8</code> | Step Over (function ko chala do, andar mat jao) |
| <code>F9</code> | Run (agle breakpoint tak chalao)                |

Abhi in tools ko sirf install karke ghoomo. Deep use Phase 3-4 mein.

## 2.5 — Helper Tools (Chhote But Kaam Ke)

Ye chhote tools RE workflow mein bahut kaam aate hain:

| Tool                                      | Kaam                                                                                  |
|-------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Detect It Easy (DIE)</b> ★             | File kaunse language/compiler se bani, packed hai ya nahi — instant batata hai        |
| <b>PE-bear / PEview / CFF Explorer</b>    | <code>.exe</code> (PE file) ki internal structure dekhna — headers, sections, imports |
| <b>Strings</b> (Sysinternals)             | File mein chhupe readable text nikaalna (URLs, passwords, messages)                   |
| <b>HxD (Hex Editor)</b>                   | File ko raw bytes mein dekhna/edit karna                                              |
| <b>Dependency Walker / PE imports</b>     | Program kaunsi DLL/functions use karta hai                                            |
| <b>Process Monitor / Process Explorer</b> | Chalte hue program ki activity (files, registry, network)                             |

### PE File Structure (Windows .exe) — Basic

Windows executables "PE format" mein hote hain. Iske parts: - **DOS Header** — purana compatibility header (`MZ` se shuru). - **PE Header** — asli metadata (32/64 bit, entry point address). - **Sections:** - `.text` — actual code (instructions) - `.data` — initialized data (variables jinki value pehle se hai) - `.rdata` — read-only data (strings, constants) - `.idata` — imports (kaunse DLL/functions use ho rahe) - `.rsrc` — resources (icons, images)

`.text` mein code hota hai (yahan reverse karte hain), `.rdata` mein strings (yahan clues milte hain). Linux mein iska equivalent "ELF format" hota hai.

## 2.6 — Pehla Practical (Zaroor Karo)

1. Ek simple C program khud likho (jaise password check karne wala).
2. Compile karke `.exe` banao.
3. **DIE** mein kholo — dekho kya batata hai (compiler, bits).
4. **Ghidra** mein import karo, analyze karo.
5. `main` function dhoondo, decompiler mein padho — apna hi likha logic pehchano!

6. **x64dbg** mein kholo, `main` pe breakpoint lagao, `F8` se step karo.

Apna hi program reverse karna best starting point hai — kyunki tumhe pehle se pata hai andar kya hai, toh tools samajhna aasan.

## Key Takeaways (Ek Line Mein)

1. **Disassembler** = assembly (exact); **Decompiler** = C-jaisa code (aasan but approximate). Dono use karo.
2. **Ghidra** (free, decompiler ke saath) = best beginner tool for static analysis.
3. **x64dbg** = best free Windows debugger for dynamic analysis.
4. Ghidra mein **rename** aur **XREF** (cross-references) sabse useful features.
5. **DIE** se file identify karo (packed? kaunsa compiler?).
6. Windows `.exe` = **PE format**; code `.text` section mein, strings `.rdata` mein.
7. Debugger keys: `F2` breakpoint, `F7` step into, `F8` step over, `F9` run.

✅ **Phase 2 Complete Jab:** Tum Ghidra aur x64dbg install karke ek program khol sako, functions dhoond sako, decompiler padh sako, aur basic navigate kar sako. Ab asli analysis shuru (Phase 3)!



# Phase 2 — Interview Questions & Answers (Hinglish)

---

RE Tools se related important interview questions, simple Hinglish jawaab ke saath.

## Q1. Disassembler aur Decompiler mein kya farak hai?

**Ans: Disassembler** machine code ko assembly mein badalta hai — ye 100% exact hota hai (jaise IDA, objdump). **Decompiler** us code ko C-jaise high-level code mein badalta hai — padhna aasan hota hai but approximate hota hai, variable naam ajeeb aur kabhi galti bhi (jaise Ghidra decompiler, Hex-Rays). Best strategy: dono use karo.

## Q2. Ghidra kya hai aur beginners ke liye kyun accha hai?

**Ans:** Ghidra ek free, powerful RE tool hai jise NSA ne banaya. Ye disassembler aur decompiler dono deta hai — matlab assembly ke saath C-jaisa code bhi dikhata hai, jo padhna aasan hai. Free hone aur decompiler dene ki wajah se beginners ke liye best hai.

## Q3. Debugger kya hota hai aur wo disassembler se kaise alag hai?

**Ans: Disassembler** program ko chalayе bina uska code dikhata hai (static analysis). **Debugger** program ko **chalate hue** rok-rok ke dekhne deta hai — breakpoints lagana, ek-ek line chalana, live register aur memory values dekhna (dynamic analysis). Example: x64dbg, GDB.

## Q4. Static analysis aur dynamic analysis mein kya farak hai?

**Ans: Static analysis** program ko chalayе bina, sirf code padh ke samajhna hai (Ghidra se). **Dynamic analysis** program ko chalte hue, runtime pe values dekhkar samajhna hai (x64dbg se). Dono complementary hain — often dono milaake use karte hain.

## Q5. Cross-reference (XREF) kya hota hai aur kyun useful hai?

**Ans:** XREF batata hai ki koi function ya variable code mein **kahan-kahan use** hua hai. Jaise agar tumhe "password" string mili, toh uske XREF se pata chalega ki wo string kaunse function mein use ho rahi — seedha important logic tak pahunch jaate ho. RE mein clues follow karne ka ye best tarika hai.

## Q6. PE file format kya hai?

**Ans:** PE (Portable Executable) Windows executables ( `.exe` , `.dll` ) ka format hai. Ismein DOS header, PE header (32/64-bit info, entry point), aur sections hote hain jaise `.text` (code), `.data` (variables), `.rdata` (strings/constants), `.idata` (imports), `.rsrc` (resources). Linux ka equivalent ELF format hai.

---

## Q7. `.text` aur `.rdata` sections mein kya hota hai?

**Ans:** `.text` section mein program ka actual code (instructions) hota hai — yahan reverse karte hain. `.rdata` section mein read-only data hota hai jaise strings aur constants — yahan often passwords, URLs, error messages jaise clues milte hain.

---

## Q8. "Strings" tool kya karta hai aur RE mein kyun kaam aata hai?

**Ans:** Strings tool file ke andar chhupe readable text (printable characters) nikaal deta hai. RE mein ye pehla step hota hai — often hardcoded passwords, URLs, API keys, error messages, ya hints seedha strings mein mil jaate hain, bina deep analysis ke.

---

## Q9. Packed binary kya hota hai aur use kaise detect karte hain?

**Ans:** Packed binary wo hai jiska code compress ya encrypt kar diya gaya hai (jaise UPX packer se), taaki analysis mushkil ho. Aisi file mein strings kam dikhte hain aur code samajh nahi aata. Ise **Detect It Easy (DIE)** jaise tool se detect karte hain — wo batata hai file packed hai ya nahi aur kaunse packer se.

---

## Q10. Ghidra mein rename karna kyun important hai?

**Ans:** Decompiler variables ko ajeeb naam deta hai jaise `local_8` , `uVar1` . Jaise-jaise hum samajhte hain ki kaunsa variable kya kar raha, unhe meaningful naam de dete hain (jaise `user_input` , `password_check` ). Isse code padhna aur logic samajhna bahut aasan ho jaata hai — ye "map banane" jaisa hai.

---

## Q11. x64dbg ke basic controls (F7, F8, F9) kya karte hain?

**Ans:** - **F2** — breakpoint lagana/hatana (program yahan ruk jaayega). - **F7** — Step Into (function ke andar chale jaao). - **F8** — Step Over (function ko chala do, but andar mat jaao). - **F9** — Run (agle breakpoint tak chalao).

---

## **Q12. Kaunse tools se RE shuru karna best hai aur kyun?**

**Ans: Ghidra + x64dbg** — dono free hain aur milkar 90% kaam kar dete hain. Ghidra static analysis (code padhna, decompiler) ke liye aur x64dbg dynamic analysis (runtime debugging) ke liye. Baad mein IDA, Binary Ninja jaise advanced tools try kar sakte ho.

# Phase 2 — Interview Questions & Answers (English)

---

The same RE Tools interview questions, answered in clear English for formal interviews.

---

## Q1. What is the difference between a disassembler and a decompiler?

**Ans:** A **disassembler** converts machine code into assembly — it's 100% exact (e.g., IDA, objdump). A **decompiler** converts that code into C-like high-level code — easier to read but approximate, with odd variable names and occasional mistakes (e.g., Ghidra's decompiler, Hex-Rays). Best practice is to use both.

---

## Q2. What is Ghidra and why is it good for beginners?

**Ans:** Ghidra is a free, powerful RE tool developed by the NSA. It provides both a disassembler and a decompiler — showing assembly alongside readable C-like code. Being free and including a decompiler makes it ideal for beginners.

---

## Q3. What is a debugger and how does it differ from a disassembler?

**Ans:** A **disassembler** shows a program's code without running it (static analysis). A **debugger** lets you inspect a program while it **runs** — setting breakpoints, stepping through instructions, and viewing live register and memory values (dynamic analysis). Examples: x64dbg, GDB.

---

## Q4. What is the difference between static and dynamic analysis?

**Ans:** **Static analysis** means understanding a program by reading its code without running it (using Ghidra). **Dynamic analysis** means understanding it while running it and observing runtime values (using x64dbg). They're complementary and often used together.

---

## Q5. What is a cross-reference (XREF) and why is it useful?

**Ans:** An XREF tells you where a function or variable is **used** in the code. For example, if you find a "password" string, its XREFs reveal which function uses it — leading you straight to the important logic. It's the best way to follow clues in RE.

---

## Q6. What is the PE file format?

**Ans:** PE (Portable Executable) is the format for Windows executables ( `.exe` , `.dll` ). It contains a DOS header, a PE header (32/64-bit info, entry point), and sections such as `.text` (code), `.data` (variables), `.rdata` (strings/constants), `.idata` (imports), and `.rsrc` (resources). The Linux equivalent is the ELF format.

---

## Q7. What is stored in the `.text` and `.rdata` sections?

**Ans:** The `.text` section holds the program's actual code (instructions) — where reversing happens. The `.rdata` section holds read-only data such as strings and constants — often revealing clues like passwords, URLs, and error messages.

---

## Q8. What does the "Strings" tool do and why is it useful in RE?

**Ans:** The Strings tool extracts readable text (printable characters) from a file. In RE it's often the first step — hardcoded passwords, URLs, API keys, error messages, or hints frequently appear directly in the strings, without deep analysis.

---

## Q9. What is a packed binary and how do you detect it?

**Ans:** A packed binary has its code compressed or encrypted (e.g., by UPX) to make analysis harder. Such files show few readable strings and unclear code. You detect them with a tool like **Detect It Easy (DIE)**, which reports whether a file is packed and by which packer.

---

## Q10. Why is renaming important in Ghidra?

**Ans:** The decompiler gives variables generic names like `local_8` , `uVar1` . As you understand what each does, you rename them to meaningful names (like `user_input` , `password_check` ). This makes the code far easier to read and reason about — it's like drawing a map as you explore.

---

## Q11. What do the basic x64dbg controls (F7, F8, F9) do?

**Ans:** - **F2** — set/remove a breakpoint (the program pauses here). - **F7** — Step Into (enter the called function). - **F8** — Step Over (execute the function without entering it). - **F9** — Run (continue to the next breakpoint).

---



## **Q12. Which tools are best to start RE with and why?**

**Ans: Ghidra + x64dbg** — both are free and together handle about 90% of the work. Ghidra for static analysis (reading code, decompiler) and x64dbg for dynamic analysis (runtime debugging). Later you can try advanced tools like IDA or Binary Ninja.

## PHASE 3

# Static Analysis

Bina chalaaye code padh ke samajhna — strings, functions, decompiler

## Phase 3 — Static Analysis (Bina Chalaaye Samajhna)

**Goal:** Program ko **chalaaye bina**, sirf code padh ke samajhna. Jaise book padhna bina act kiye. Ye RE ka core skill hai. Time: **3-4 hafte**.

### 3.0 — Static Analysis Kya Hai?

**Static analysis** matlab program ko run kiye bina analyze karna — bas uska code (assembly / decompiled C) padhkar samajhna ki wo kya karega.

**Fayde:** - ☒ Safe hai — malware bhi chalta nahi, isliye PC surakshit. - ☒ Poora code ek saath dikh jaata hai. - ☒ Chhupe paths bhi dikhte hain (jo shayad kabhi run na hon).

**Nuksan:** - ☒ Encrypted/packed code samajhna mushkil. - ☒ Runtime values (jaise decrypt hua password) nahi dikhte.

Isiliye static + dynamic dono seekhte hain. Static se "kya karega" samajho, dynamic (Phase 4) se "kya kar raha hai" confirm karo.

### 3.1 — Step 1: File Ko Pehchano (Recon)

Analysis se pehle file ke baare mein basic info nikaalo:

1. **File type?** — Windows PE? Linux ELF? 32-bit ya 64-bit? (DIE tool se)

2. **Packed/encrypted?** — Agar packed hai toh pehle unpack karna padega (DIE batata hai).
3. **Compiler/language?** — C, C++, Go, Rust, .NET? (har ka reverse alag).
4. **Strings nikaalo** — sabse pehla clue yahan milta hai.

## Strings — Sabse Powerful Pehla Kadam

```
strings program.exe          # sab readable text
strings program.exe | grep -i "pass"  # "password" jaisa dhoondo
```

Aksar milte hain: - Passwords, secret keys (hardcoded) - URLs, IP addresses (malware C2 servers) - Error/success messages ("Wrong password!", "Access granted!") - File paths, registry keys

**Trick:** "Success" ya "Wrong" message dhoondo. Uske XREF se seedha password-check function mil jaata hai!

## 3.2 — Step 2: Entry Point Aur main() Dhoondo

Program kahan se shuru hota hai? - **Entry point** — technically program yahan se chalu hota hai (runtime setup code). - **main()** — asli logic yahan hota hai. Ghidra usually `main` dhoond ke label kar deta hai.

Ghidra mein: Symbol Tree → Functions → `main` (ya `entry`, `WinMain`). Wahan se logic follow karo.

## 3.3 — Step 3: Functions Ka Flow Follow Karo

Ek program bahut se functions ka jaal hota hai. Unhe samajhne ka tarika:

1. `main` se shuru karo.
2. Dekho `main` kaunse functions ko `call` karta hai.
3. Har important function ko rename karo (jaise `FUN_00401000` → `check_password`).
4. Chhote helper functions pehle samjho, phir bade.

## Call Graph

Ghidra ek "call graph" bana sakta hai — dikhata hai kaunsa function kise call karta hai. Isse poore program ka nakshaa mil jaata hai.

## 3.4 — Step 4: Control Flow Samajhna (If, Loops)

Decompiled/assembly code mein logic ko pehchano:

### If-else pehchano

```
// Decompiled Ghidra output aisa dikhta hai:
if (iVar1 == 0x2a) {           // agar iVar1 == 42
    puts("Correct!");
} else {
    puts("Wrong!");
}
```

Assembly mein yahi `cmp` + `je` / `jne` hota hai (Phase 1 wala).

### Loops pehchano

```
for (i = 0; i < 10; i = i + 1) {
    sum = sum + i;
}
```

Assembly mein loop ek "backward jump" se banta hai (upar wali line pe wapas kudna).

### Control Flow Graph (CFG)

Ghidra CFG banata hai — boxes (basic blocks) aur arrows (jumps). Isse dekho code kaise flow karta hai — kaunsi condition kahan le jaati hai.

## 3.5 — Step 5: Decompiler Ka Sahi Use

Ghidra ka decompiler assembly ko C-jaisa dikhata hai. Ye padhna aasan hai, but kuch baatein yaad rakho:

- Variable naam ajeeb honge: `local_8`, `iVar1`, `uVar2`, `param_1`. Inhe **rename** karo jaise samajhte jao.
- Types galat ho sakte hain (int vs pointer). Ghidra ko sahi type batao (`Ctrl+L`).
- Kabhi decompiler galti karta hai — doubt ho toh assembly (Listing window) dekho.

## Common function names pehchano

Ye standard library functions dikhein toh turant matlab samjho: | Function | Matlab | |-----|-----| | `strcmp` / `strncmp` | do strings compare (aksar password check!) | | `strcpy` / `memcpy` | data copy | | `printf` / `puts` | screen pe print | | `scanf` / `gets` / `fgets` | user se input lena | | `malloc` / `free` | memory allocate/release | | `fopen` / `fread` | file operations |

`strcmp` dikha? 90% chance wahan koi string comparison ho raha hai — password ya key check.

## 3.6 — Real Example: Ek Password Checker Crack Karna

Chalo ek typical "crackme" static analysis se todte hain.

### Program ka behaviour

```
$ ./crackme
Enter password: hello
Wrong password!
```

### Ghidra decompiler output (typical)

```
int main(void) {
    char input[32];           // user ka input yahan aayega
    printf("Enter password: ");
    scanf("%s", input);       // user se password liya
    if (strcmp(input, "Sup3rS3cr3t") == 0) { // compare!
        puts("Access granted!");
        return 0;
    }
    puts("Wrong password!");
    return 1;
}
```

**Line by line samjho:** - `char input[32];` — 32 characters ka box, user input ke liye. - `scanf("%s", input);` — user se password padha, `input` mein daala. - `strcmp(input, "Sup3rS3cr3t")` — user ka input "Sup3rS3cr3t" se compare kiya. `== 0` matlab dono barabar (strcmp equal pe 0 deta hai). - Agar barabar → "Access granted!", warna "Wrong password!".

**Password mil gaya:** `Sup3rS3cr3t` — bina program chalaye, sirf code padh ke! Ye static analysis ki power hai.

### Thoda mushkil version (obfuscated)

Kabhi password seedha nahi hota — thoda "encode" hota hai:

```
// har character ko XOR kiya gaya 0x10 se
char secret[] = {0x63, 0x75, 0x64, ...}; // encoded bytes
for (i = 0; i < len; i++) {
    if ((input[i] ^ 0x10) != secret[i]) { // input ko bhi XOR karke check
        // wrong
    }
}
```

Yahan tumhe khud `secret` bytes ko `0x10` se XOR karke asli password nikaalna padega. (Ya dynamic analysis se dekh lo — Phase 4.)

## 3.7 — Static Analysis Workflow (Summary)

1. Recon → DIE se file type, packed check
2. Strings → clues (passwords, messages, URLs)
3. Entry → main() dhoondo
4. Flow → functions follow karo, rename karo
5. Logic → if/loops/strcmp pehchano
6. Solve → password/key/logic nikaalo
7. Notes → comments aur renames karte raho



## Phase 3 — Hands-On Practice

- crackmes.one se **easy** password-based crackmes lo.
- Sirf static analysis (Ghidra) se solve karne ki koshish karo.
- Har function ko rename + comment karo.
- Kam se kam 3-4 crackmes static se solve karo.

## Key Takeaways (Ek Line Mein)

1. Static analysis = program **run kiye bina** code padh ke samajhna (safe + complete view).
2. **Strings pehla step** — passwords, messages, URLs aksar seedha mil jaate hain.
3. `main()` se shuru karo, functions ko follow aur rename karte jao.
4. `cmp` + jump = if-else; backward jump = loop.
5. `strcmp` dikha = string comparison (aksar password check).
6. Decompiler aasan hai but approximate — doubt pe assembly dekho.
7. Encrypted password ko XOR/decode karke nikaalna pad sakta hai.

✓ **Phase 3 Complete Jab:** Tum ek simple program ka decompiled code padh ke samajh sako ki wo kya logic use kar raha, aur ek easy crackme static analysis se solve kar sako. Ab dynamic analysis (Phase 4)! 🚀

# Phase 3 — Interview Questions & Answers (Hinglish)

---

Static Analysis se related important interview questions, simple Hinglish jawaab ke saath.

---

## Q1. Static analysis kya hai?

**Ans:** Static analysis matlab program ko **chalaye bina** analyze karna — sirf uska code (assembly ya decompiled C) padhkar samajhna ki wo kya karega. Ye safe hota hai (malware bhi chalta nahi) aur poora code ek saath dikh jaata hai.

---

## Q2. Static analysis ke fayde aur nuksan kya hain?

**Ans:** - **Fayde:** Safe hai (kuch execute nahi hota), poora code aur saare code paths dikhte hain (chhupe hue bhi). - **Nuksan:** Encrypted/packed code samajhna mushkil, aur runtime pe banne wali values (jaise decrypt hua password) nahi dikhtin.

Isliye ise dynamic analysis ke saath milaake use karte hain.

---

## Q3. Kisi program ka analysis shuru karte waqt pehla step kya hota hai?

**Ans:** File ka **recon** — file type (PE/ELF, 32/64-bit), packed hai ya nahi (DIE tool se), aur kaunse compiler/language se bani. Uske baad **strings** nikaalna, jahan aksar passwords, messages, aur URLs jaise clues seedha mil jaate hain.

---

## Q4. Strings analysis se kya milta hai?

**Ans:** File ke andar chhupe readable text — jaise hardcoded passwords/keys, URLs ya IP addresses (malware ke C2 servers), success/error messages ("Access granted!", "Wrong password!"), file paths, aur registry keys. Ye pehla aur sabse aasan clue source hai.

---

## Q5. Entry point aur main() mein kya farak hai?

**Ans:** **Entry point** wo jagah hai jahan se program technically chalu hota hai — ismein runtime setup code hota hai. **main()** wo function hai jahan program ka asli logic likha hota hai. RE mein hum usually main() se analysis shuru karte hain.

---



## Q6. Program ka control flow static analysis mein kaise pehchante hain?

**Ans:** If-else `cmp` + conditional jump ( `je` / `jne` ) se bante hain, aur loops ek backward jump (upar wali line pe wapas kudna) se. Decompiler ismein `if`, `for`, `while` jaisa dikha deta hai. Ghidra ka Control Flow Graph (CFG) boxes aur arrows se ye flow visually dikhata hai.

---

## Q7. `strcmp` function dikhe toh kya samajhna chahiye?

**Ans:** `strcmp` do strings ko compare karta hai aur barabar hone pe `0` return karta hai. RE mein jab `strcmp(input, "kuch") == 0` dikhe, to samajh jao wahan koi string comparison ho raha hai — aksar password ya key check. Ye function seedha logic tak le jaata hai.

---

## Q8. Decompiler output padhte waqt kaunsi baatein dhyaan mein rakhni chahiye?

**Ans:** Variable naam ajeeb honge ( `local_8`, `iVar1`, `param_1` ) — unhe samajhte hue rename karo. Types galat ho sakte hain (int vs pointer) — sahi type set karo. Aur decompiler kabhi galti karta hai — doubt ho to assembly (Listing) dekho.

---

## Q9. Rename aur comment karna static analysis mein kyun zaroori hai?

**Ans:** Decompiler generic naam deta hai jo samajhna mushkil hai. Jaise-jaise logic samajhte ho, functions/variables ko meaningful naam do aur notes (comments) likho. Ye ek "map banane" jaisa hai — jitna map banega, poora program utna clear hota jaayega.

---

## Q10. Agar password seedha `strcmp` mein na ho, encoded ho, tab kya karoge?

**Ans:** Aksar password XOR ya kisi simple encoding se chhupaya hota hai. Tab code se encoding logic samajhkar (jaise har byte ko `0x10` se XOR) usko ulta karke asli password nikaalna padta hai. Ya dynamic analysis mein breakpoint lagaakar decrypt hui value memory mein dekh lete hain.

---

## Q11. Cross-references (XREF) static analysis mein kaise madad karte hain?

**Ans:** XREF batata hai koi string ya function kahan use hua. Jaise agar "Wrong password!" string mili, uske XREF se seedha wo function mil jaata hai jahan password check ho raha hai. Isse important logic tak jaldi pahunch jaate hain bina poora code padhe.

---

## **Q12. Packed binary static analysis mein problem kyun banta hai?**

**Ans:** Packed binary ka asli code compress/encrypt hota hai, isliye static mein sirf packer ka code aur garbage dikhta hai — asli logic aur strings chhupe hote hain. Ise pehle unpack karna padta hai (ya dynamic analysis mein memory se dump karna padta hai) tab hi asli code analyze ho paata hai.

# Phase 3 — Interview Questions & Answers (English)

---

The same Static Analysis interview questions, answered in clear English for formal interviews.

---

## Q1. What is static analysis?

**Ans:** Static analysis means analyzing a program **without running it** — understanding what it will do by reading its code (assembly or decompiled C). It's safe (even malware never executes) and shows the whole code at once.

---

## Q2. What are the advantages and disadvantages of static analysis?

**Ans:** - **Advantages:** It's safe (nothing executes), and you see the entire code and all code paths (even hidden ones). - **Disadvantages:** Encrypted/packed code is hard to read, and runtime-generated values (like a decrypted password) aren't visible.

That's why it's combined with dynamic analysis.

---

## Q3. What is the first step when starting to analyze a program?

**Ans:** File **recon** — identify the file type (PE/ELF, 32/64-bit), whether it's packed (using a tool like DIE), and which compiler/language produced it. Then extract **strings**, which often directly reveal clues like passwords, messages, and URLs.

---

## Q4. What do you get from strings analysis?

**Ans:** Readable text hidden in the file — hardcoded passwords/keys, URLs or IP addresses (malware C2 servers), success/error messages ("Access granted!", "Wrong password!"), file paths, and registry keys. It's the first and easiest source of clues.

---

## Q5. What is the difference between the entry point and main()?

**Ans:** The **entry point** is where the program technically begins — it contains runtime setup code. **main()** is the function containing the program's actual logic. In RE we usually start analysis from main().

---

## Q6. How do you recognize control flow in static analysis?

**Ans:** If-else is formed by `cmp` plus a conditional jump ( `je` / `jne` ), and loops by a backward jump (jumping back to an earlier line). The decompiler shows these as `if` , `for` , `while` . Ghidra's Control Flow Graph (CFG) displays this flow visually as boxes and arrows.

---

## Q7. What should you infer when you see the `strcmp` function?

**Ans:** `strcmp` compares two strings and returns `0` when they're equal. In RE, seeing `strcmp(input, "something") == 0` signals a string comparison — often a password or key check. It leads you straight to the important logic.

---

## Q8. What should you keep in mind while reading decompiler output?

**Ans:** Variable names will be generic ( `local_8` , `iVar1` , `param_1` ) — rename them as you understand them. Types may be wrong (int vs pointer) — set the correct type. And the decompiler occasionally makes mistakes — when in doubt, check the assembly (Listing view).

---

## Q9. Why are renaming and commenting important in static analysis?

**Ans:** The decompiler assigns generic names that are hard to follow. As you understand the logic, give functions/variables meaningful names and add notes (comments). It's like drawing a map — the more you map, the clearer the whole program becomes.

---

## Q10. What do you do if the password isn't in a plain `strcmp` but is encoded?

**Ans:** Passwords are often hidden with XOR or simple encoding. You then read the encoding logic (e.g., each byte XOR-ed with `0x10` ) and reverse it to recover the real password. Alternatively, in dynamic analysis, set a breakpoint and read the decrypted value directly from memory.

---

## Q11. How do cross-references (XREFs) help in static analysis?

**Ans:** XREFs show where a string or function is used. For example, if you find the "Wrong password!" string, its XREF leads directly to the function performing the password check. This gets you to the important logic quickly without reading all the code.

---

### **Q12. Why is a packed binary a problem for static analysis?**

**Ans:** A packed binary's real code is compressed/encrypted, so static analysis shows only the packer's stub and garbage — the real logic and strings are hidden. It must first be unpacked (or dumped from memory during dynamic analysis) before the real code can be analyzed.

## PHASE 4

# Dynamic Analysis

Chalate hue samajhna – breakpoints, live values, patching

## Phase 4 — Dynamic Analysis (Chalate Hue Samajhna)

**Goal:** Program ko **chalate hue** rok-rok ke samajhna. Jab static se samajh na aaye (ya code chhupa ho), tab ye kaam aata hai. Time: **3-4 hafte**.

### 4.0 — Dynamic Analysis Kya Hai?

**Dynamic analysis** matlab program ko **actually chalakar**, beech mein rok-rok ke, live values dekhkar samajhna. Static "book padhna" tha, dynamic "movie dekhna" hai — cheezein hoti hui dikhti hain.

**Kab use karein:** - Code encrypted/packed hai (static mein samajh nahi aata). - Password runtime pe decrypt/generate hota hai. - Bahut complex logic — chala ke dekhna aasan hai.

⚠ **Safety:** Malware ya unknown files hamesha **Virtual Machine (VM)** mein chalao, apne main PC pe kabhi nahi. (VM setup Phase 5 mein.)

### 4.1 — Debugger Ki Power

Debugger program ko control mein leta hai. Tum decide karte ho kab chale, kab ruke, aur har point pe sab kuch dekh sakte ho.

#### Basic Concepts

**Breakpoint** — ek nishaan jahan program apne aap ruk jaayega. Sabse important tool. - Software breakpoint ( **F2** in x64dbg ) — kisi instruction pe. - Conditional breakpoint — sirf jab koi condition

sachh ho (jaise `eax == 5`). - Hardware breakpoint — memory read/write pe ruk jaana.

**Stepping** — ek-ek karke chalana: | Action | x64dbg | GDB | Kaam | |-----|-----|-----|-----| | Step Into | `F7` | `stepi` / `s` | function ke andar ghuso | | Step Over | `F8` | `nexti` / `n` | function ko chala do, andar mat jao | | Step Out | `Ctrl+F9` | `finish` | current function se bahar niklo | | Run/Continue | `F9` | `continue` / `c` | agle breakpoint tak chalao |

**Watch/Inspect** — chalte hue dekhna: - **Registers** — live values (eax, eip...). - **Memory/Dump** — kisi address pe kya rakha hai (hex + ASCII). - **Stack** — abhi stack pe kya hai.

## 4.2 — Runtime Values Pakadna (Sabse Bada Fayda)

Static mein bas code dikhta hai, dynamic mein **asli values** dikhti hain.

**Example scenario:** Ek program password ko encrypt karke compare karta hai. Static analysis mein tumhe sirf encrypted bytes dikhenge — asli password nahi. Lekin:

1. `strcmp` (ya jahan comparison hota hai) pe **breakpoint** lagao.
2. Program chalao (`F9`), password wahan pe rukega.
3. Registers/stack dekho — `strcmp` ke arguments mein **decrypt hua asli password** memory mein dikhega! 🕵️

Ye dynamic analysis ka superpower hai — encryption ke baad ki asli value seedha memory mein dekh lena, bina encryption samjhe.

### Kahan breakpoint lagayein? (common spots)

- `strcmp` / `strncmp` / `memcmp` — comparisons pe.
- `scanf` / `fgets` ke baad — input padhne ke baad.
- Interesting function ke shuru mein.
- Jahan "success/wrong" message print hone se pehle decision hota hai.

## 4.3 — Patching (Code Badalna)

**Patching** matlab program ke bytes/instructions ko badal ke uska behaviour change karna. Ye RE ka bahut powerful (aur maze wala) hissa hai.

### Classic Example: "Wrong" ko "Right" bana dena

Password check aisa hota hai:

```

cmp eax, ebx      ; password sahi hai?
jne wrong_password ; agar NAHI (not equal), wrong pe jao
; ... access granted ...
wrong_password:
; ... wrong message ...

```

**Patch idea:** `jne` (jump if not equal) ko `je` (jump if equal) ya `nop` (kuch mat karo) mein badal do. Ab galat password pe bhi "access granted" chalega!

```

cmp eax, ebx
je wrong_password ; ULTA kar diya (ya nop se hata do)

```

## Patching ke tarike

- **NOP karna** — instruction ko `nop` (No Operation, `0x90`) se replace karna — matlab "kuch mat karo, aage badho".
- **Jump ulta karna** — `je` ↔ `jne`, `jg` ↔ `jle` etc.
- **Return value badalna** — function ko forcefully `return 1` (ya 0) karwana.

## x64dbg mein patch kaise

1. Jis instruction ko badalna hai uspe right-click → "Assemble" ( `Space` ).
2. Nayi instruction likho (jaise `nop` ya `je` ).
3. File → "Patch file" → save karke naya `.exe` banao.

⚠ Patching sirf learning ya authorized testing ke liye. Kisi ki software ki protection todna illegal hai.

## 4.4 — Static + Dynamic Combo (Real Workflow)

Asli RE mein dono milaake use hote hain:



1. Static (Ghidra) → overall structure samjho, interesting functions dhoondo
2. Breakpoints → un functions pe x64dbg mein breakpoint lagao
3. Dynamic (x64dbg) → chala ke live values, decrypt data dekho
4. Wapas Static → nayi samajh ke saath code dubara padho
5. Patch (optional) → behaviour change karke test karo

Ye loop repeat hota rehta hai. Ek se clue milta hai, doosre se confirm hota hai.

## 4.5 — Anti-Debugging (Basic Idea)

Kuch programs (khaaskar malware) detect kar lete hain ki debugger laga hai, aur phir alag behave karte hain ya band ho jaate hain — taaki analysis mushkil ho.

**Common anti-debug tricks:** - `IsDebuggerPresent()` — Windows API jo batata hai debugger hai ya nahi. - Timing checks — code kitni der mein chala (debugger mein slow hota hai). - PEB flag check — process ke internal flag dekhna.

**Bypass (basic):** - `IsDebuggerPresent` ke return value ko patch karke 0 (no debugger) bana do. - Anti-debug check ko `nop` se hata do. - Plugins (jaise ScyllaHide) automatically kai anti-debug tricks chhupa dete hain.

Ye advanced topic hai. Abhi bas idea rakho ki aisa hota hai. Deep dive baad mein.

## 4.6 — Dynamic Example: Encrypted Password Nikaalna

**Scenario:** Program password ko reverse + XOR karke compare karta hai. Static mushkil.

**Steps:** 1. Ghidra mein dekha — comparison `memcmp` pe ho raha hai. 2. x64dbg mein program kholo. 3. `memcmp` pe breakpoint ( `bp memcmp` ya address pe `F2` ). 4. Program chalao, koi bhi password daalo. Breakpoint hit hoga. 5. `memcmp` ke arguments (registers/stack) dekho — ek argument mein tumhara input, doosre mein **asli expected password** (already decrypt hokar). 6. Memory dump mein wo asli password padho. Done! 🎯

Encryption ki logic samajhne ki zaroorat hi nahi padi — dynamic ne decrypt hui value seedha dikha di.



## Phase 4 — Hands-On Practice

- Wahi crackmes (Phase 3 wale) ab **dynamic** analysis se solve karo.
- Ek crackme mein **patch** karke "wrong" ko "access granted" bana ke dekho.
- `strcmp` / `memcmp` pe breakpoint laga ke expected value pakdo.
- Static aur dynamic dono ka combo practice karo.



## Key Takeaways (Ek Line Mein)

1. Dynamic analysis = program **chala ke**, rok-rok ke, live values dekhna.
2. **Breakpoint** sabse important — jahan chaho program ruk jaaye.
3. `F7` step into, `F8` step over, `Ctrl+F9` step out, `F9` run.
4. Comparison functions ( `strcmp` / `memcmp` ) pe breakpoint = asli password memory mein dikh jaata hai.
5. **Patching** = instructions badalna ( `nop` , jump ulta) — behaviour change.
6. Static + dynamic ko **milaake** use karo — ek se clue, doosre se confirm.
7. **Anti-debugging** hoti hai — programs debugger detect karte hain; basic bypass patching/plugins se.
8. Unknown files hamesha **VM** mein.

✅ **Phase 4 Complete Jab:** Tum x64dbg/GDB mein breakpoint laga sako, live values/memory dekh sako, ek simple patch kar sako, aur ek crackme dynamic analysis se solve kar sako. Ab asli maidan — practice (Phase 5)! 🚀

# Phase 4 — Interview Questions & Answers (Hinglish)

---

Dynamic Analysis se related important interview questions, simple Hinglish jawaab ke saath.

## Q1. Dynamic analysis kya hai?

**Ans:** Dynamic analysis matlab program ko **actually chalakar**, beech-beech mein rok-rok ke, live values dekhkar samajhna. Static "book padhne" jaisa tha, dynamic "movie dekhne" jaisa hai — cheezein hoti hui dikhti hain, jaise register aur memory ki actual values.

## Q2. Dynamic analysis kab zaroori ho jaata hai?

**Ans:** Jab code encrypted ya packed ho (static mein samajh na aaye), jab password/key runtime pe decrypt ya generate hota ho, ya jab logic itna complex ho ki chala ke dekhna aasan ho. Aisi situations mein static kaafi nahi hota.

## Q3. Breakpoint kya hota hai?

**Ans:** Breakpoint ek nishaan hai jahan program apne aap ruk jaata hai, taaki hum us point pe registers, memory aur stack dekh sakein. Ye debugger ka sabse important tool hai. Types: software, conditional (jab koi condition sach ho), aur hardware (memory read/write pe).

## Q4. Step Into, Step Over aur Step Out mein kya farak hai?

**Ans:** - **Step Into (F7):** agar current line ek function call hai, uske andar ghus jao. - **Step Over (F8):** function ko poora chala do, but uske andar mat jao. - **Step Out (Ctrl+F9):** current function se bahar nikal jao (jahan se call hua tha).

## Q5. Dynamic analysis ka sabse bada fayda kya hai?

**Ans:** Runtime pe **asli values** dikh jaati hain. Jaise agar password encrypted compare hota hai, static mein sirf encrypted bytes dikhenge, lekin comparison function pe breakpoint lagaakar hum decrypt hui asli value seedha memory mein dekh sakte hain — bina encryption samjhe.

## Q6. Password nikaalne ke liye breakpoint kahan lagana chahiye?

**Ans:** Comparison functions pe — `strcmp`, `strncmp`, `memcmp`. Yahan program tumhare input ko expected (asli) value se compare karta hai, aur us waqt dono values (tumhara input + asli password) memory/registers mein dikh jaati hain. Input padhne wale functions (`scanf` / `fgets`) ke baad bhi useful hota hai.

---

## Q7. Patching kya hoti hai?

**Ans:** Patching matlab program ke instructions/bytes ko badal ke uska behaviour change karna. Classic example: password check ke `jne` (jump if not equal) ko `je` ya `nop` mein badal dena — jisse galat password pe bhi "access granted" chalne lage.

---

## Q8. NOP instruction kya karti hai aur patching mein kaise use hoti hai?

**Ans:** `nop` (No Operation, byte `0x90`) ka matlab "kuch mat karo, agli instruction pe jao". Patching mein hum kisi unwanted instruction (jaise ek check ya jump) ko `nop` se replace kar dete hain, taaki wo effectively hat jaaye aur program aage badh jaaye.

---

## Q9. Static aur dynamic analysis ko saath mein kaise use karte hain?

**Ans:** Ek real workflow: pehle static (Ghidra) se overall structure aur interesting functions dhoondo, phir un functions pe dynamic (x64dbg) mein breakpoint lagaakar live values/decrypt data dekho, phir nayi samajh ke saath wapas static code padho. Ye loop repeat hota hai — ek se clue milta hai, doosre se confirm hota hai.

---

## Q10. Anti-debugging kya hota hai?

**Ans:** Ye techniques hain jinke program detect kar leta hai ki uspe debugger laga hai, aur phir alag behave karta hai ya band ho jaata hai — taaki analysis mushkil ho. Common tricks:

`IsDebuggerPresent()` API, timing checks (code slow chala toh debugger hai), aur PEB flags dekhna.

---

## Q11. Anti-debugging ko bypass kaise karte hain (basic)?

**Ans:** `IsDebuggerPresent` jaise check ke return value ko patch karke 0 (no debugger) bana dete hain, ya anti-debug check ko `nop` se hata dete hain. Ghidra/x64dbg ke plugins (jaise ScyllaHide) automatically bahut si anti-debug tricks chhupa dete hain.

---

### **Q12. Malware ka dynamic analysis karte waqt kya precaution zaroori hai?**

**Ans:** Malware ko hamesha ek isolated **Virtual Machine (VM)** mein chalo, kabhi apne main PC pe nahi. VM ka network isolate rakho aur snapshots use karo, taaki kuch bigad jaaye toh clean state pe wapas aa sako. Isse tumhara asli system aur data surakshit rehta hai.

# Phase 4 — Interview Questions & Answers (English)

---

The same Dynamic Analysis interview questions, answered in clear English for formal interviews.

---

## Q1. What is dynamic analysis?

**Ans:** Dynamic analysis means understanding a program by **actually running it**, pausing it, and observing live values. If static analysis is "reading a book," dynamic is "watching a movie" — you see things happen, such as the actual values in registers and memory.

---

## Q2. When does dynamic analysis become necessary?

**Ans:** When the code is encrypted or packed (unclear statically), when a password/key is decrypted or generated at runtime, or when the logic is so complex that running it is easier than reading it. In these cases static analysis alone isn't enough.

---

## Q3. What is a breakpoint?

**Ans:** A breakpoint is a marker where the program automatically pauses, so we can inspect registers, memory, and the stack at that point. It's the debugger's most important tool. Types include software, conditional (pause only when a condition is true), and hardware (pause on memory read/write).

---

## Q4. What is the difference between Step Into, Step Over, and Step Out?

**Ans:** - **Step Into (F7):** if the current line is a function call, enter that function. - **Step Over (F8):** execute the function fully without entering it. - **Step Out (Ctrl+F9):** run until you exit the current function (back to its caller).

---

## Q5. What is the biggest advantage of dynamic analysis?

**Ans:** You see the **actual values** at runtime. For example, if a password is compared in encrypted form, static analysis shows only encrypted bytes; but by placing a breakpoint at the comparison function, you can read the decrypted real value directly from memory — without understanding the encryption.

---

## Q6. Where should you place a breakpoint to recover a password?

**Ans:** At comparison functions — `strcmp`, `strncmp`, `memcmp`. There the program compares your input against the expected value, and at that moment both values (your input and the real password) are visible in memory/registers. Right after input functions ( `scanf` / `fgets` ) is also useful.

---

## Q7. What is patching?

**Ans:** Patching means changing a program's instructions/bytes to alter its behavior. A classic example: changing a password check's `jne` (jump if not equal) to `je` or `nop`, so that even a wrong password leads to "access granted."

---

## Q8. What does the NOP instruction do and how is it used in patching?

**Ans:** `nop` (No Operation, byte `0x90`) means "do nothing, move to the next instruction." In patching we replace an unwanted instruction (like a check or a jump) with `nop`, effectively removing it so the program continues past it.

---

## Q9. How do you use static and dynamic analysis together?

**Ans:** A real workflow: first use static (Ghidra) to find the overall structure and interesting functions, then set breakpoints on those functions in dynamic (x64dbg) to observe live values/decrypted data, then re-read the static code with new understanding. This loop repeats — one gives clues, the other confirms them.

---

## Q10. What is anti-debugging?

**Ans:** These are techniques by which a program detects that a debugger is attached and then behaves differently or exits — to make analysis harder. Common tricks: the `IsDebuggerPresent()` API, timing checks (code runs slower under a debugger), and inspecting PEB flags.

---

## Q11. How do you bypass anti-debugging (basics)?

**Ans:** Patch the return value of a check like `IsDebuggerPresent` to 0 (no debugger), or `nop` out the anti-debug check. Plugins for Ghidra/x64dbg (like ScyllaHide) automatically hide many anti-debug tricks.

---

**Q12. What precautions are essential when dynamically analyzing malware?**

**Ans:** Always run malware in an isolated **Virtual Machine (VM)**, never on your main PC. Keep the VM's network isolated and use snapshots, so you can revert to a clean state if something goes wrong. This keeps your real system and data safe.



## PHASE 5

# Real Practice

Asli maidan – crackmes, CTF, malware analysis aur VM lab

## Phase 5 — Real Practice (Asli Maidan)

**Goal:** Ab tak theory + basics ho gaye. RE **practice se** aati hai. Jitne challenges solve karoge, utne acche banoge. Time: **Ongoing — hamesha chalta rehta hai.**

### 5.0 — Practice Kyun Sabse Important?

RE swimming jaisi hai — kitni bhi books padho, jab tak paani mein na kudo, seekh nahi paoge. Phase 0-4 mein tumne "tairna" theory mein padha, ab asli paani mein utarne ka time hai.

**Golden Rule:** Har hafte kam se kam 2-3 challenges solve karo. Rukna nahi. Atak jao toh writeup dekho, samjho, aage badho.

### 5.1 — Apna Lab Setup Banao (Safe Environment)

Real practice se pehle ek surakshit environment banao, khaaskar malware ke liye.

#### Virtual Machine (VM) — Zaroori

- **VirtualBox** (free) ya **VMware** install karo.
- Ismein alag OS chalao — jaise ek **Windows VM** aur ek **Linux VM** (Kali/Ubuntu).
- Malware/unknown files **sirf VM mein**, kabhi main PC pe nahi.

## VM ke important settings

- **Snapshots** — VM ka "save point". Kuch bigad jaaye (malware ne kharaab kiya), toh ek click mein wapas clean state.
- **Network isolation** — malware ke liye internet band ya "host-only" karo, taaki wo phail na sake ya server se baat na kare.
- **Shared folder off** — malware ke waqt VM aur host ke beech folder share mat karo.

## Tools VM mein install karo

- Ghidra, x64dbg, DIE, PE-bear, HxD, Strings, Process Monitor — sab ek VM mein ready rakho ("analysis VM").

## 5.2 — Crackmes (Beginner Friendly)

**Crackmes** = specially RE practice ke liye bane programs. Legal, safe, aur difficulty-wise sorted.

### crackmes.one

- 🌐 Website pe difficulty aur quality rating hoti hai.
- **Easy / 1-2 difficulty** se shuru karo.
- Typical challenges:
- Password/serial key nikaalna
- "Registration" bypass karna
- Hidden logic samajhna

## Ek crackme solve karne ka tarika

1. Download karo (VM mein)
2. DIE se check — kaunsa language, packed?
3. Strings dekho — koi clue?
4. Ghidra mein kholo — main dhoondo, logic padho (static)
5. Atak gaye? x64dbg mein breakpoint lagao (dynamic)
6. Password/key nikaalo
7. Apna writeup likho (kaise solve kiya)

**Writeup likhna zaroori:** Jo solve kiya use likho. Isse concept pakka hota hai aur portfolio banta hai.

## 5.3 — CTF Challenges (Capture The Flag)

**CTF** = hacking competitions jismein challenges solve karke "flags" (secret strings) capture karte ho. "Reversing" category pe focus karo.

### Beginners ke liye

- 🌐 **picoCTF** — best beginner CTF, free, hamesha available (practice mode). Reversing challenges se shuru.
- 🌐 **OverTheWire** — Linux + basics.

### Guided learning platforms

- 🌐 **TryHackMe** — step-by-step, beginner friendly, guided rooms.
- 🌐 **HackTheBox** — thoda advanced, real-world jaisa.

### Live competitions

- 🌐 **CTFtime.org** — duniya bhar ki live CTFs ki list. Team banake participate karo.

CTF reversing challenges tumhe time pressure mein sochna sikhate hain — real skill isse aati hai.

## 5.4 — Malware Analysis (Advanced, Careful!)

Real virus/malware analyze karna — **sirf tab jab basics solid ho**, aur **hamesha isolated VM mein**.

### ⚠️ Safety FIRST

- **Kabhi bhi** malware main PC pe mat chalo.
- Isolated VM + no network + snapshots.
- Analysis ke baad VM ko snapshot se revert karo.

### Kya seekhte hain

- Malware kaise phailta hai (persistence, spreading).
- C2 (Command & Control) server se kaise baat karta hai.
- Kya cheezein encrypt/steal karta hai.
- IOCs (Indicators of Compromise) nikaalna — detection ke liye.

### Sample sources (extreme care ke saath)

- 🌐 **MalwareBazaar, theZoo, VirusShare** — real samples. Sirf experienced hone pe, isolated setup mein.

- Shuru mein "malware analysis practice samples" (safe, bane hue) use karo.

## Types of malware analysis

- **Static** — bina chalaye (safe): strings, imports, PE structure.
- **Dynamic** — chala ke (VM mein): behaviour dekhna (files, registry, network).

## 5.5 — Practice Routine (Consistency Is Key)

Ek achha weekly plan:

```
Din 1-2: Ek naya crackme solve karo (static + dynamic)
Din 3: Uska writeup likho
Din 4-5: Ek CTF reversing challenge (picoCTF/TryHackMe)
Din 6: Kisi aur ka writeup padho (nayi tricks seekho)
Din 7: Notes review + revise
```

Consistency > intensity. Roz thoda better, ek din bahut se behtar.

## 5.6 — Community Join Karo

Akele mat seekho — logon se bahut jaldi seekhoge: - **Reddit:** r/ReverseEngineering, r/CTF, r/asm, r/Malware - **Discord servers:** CTF teams, RE communities - **Twitter/X:** RE researchers follow karo (new tricks, writeups) - Apne solved challenges ke writeups share karo — feedback milega.

## Phase 5 — Milestones

- [ ] VirtualBox/VMware setup + analysis VM ready (snapshots ke saath)
- [ ] Pehla easy crackme solve kiya 🎉
- [ ] 5+ crackmes solve kiye (mix static/dynamic)
- [ ] Pehla picoCTF reversing challenge solve kiya
- [ ] Apna pehla writeup likha
- [ ] Ek RE community join ki

## Key Takeaways (Ek Line Mein)

1. RE **practice** se aati hai — theory kaafi nahi, challenges solve karo.
2. **VM + snapshots + network isolation** = safe lab (malware ke liye must).
3. **crackmes.one** se easy crackmes se shuru karo.
4. **picoCTF / TryHackMe** se CTF reversing seekho.
5. **Malware analysis** sirf solid basics ke baad, hamesha isolated VM mein.
6. Har challenge ka **writeup likho** — concept pakka + portfolio.
7. **Consistency** (har hafte 2-3 challenges) sabse important.
8. **Community** join karo — akele se tez seekhoge.

---

✅ **Phase 5 Ongoing:** Ye phase kabhi "complete" nahi hota — jitni practice, utne acche. Jab comfortable ho jao (bahut se challenges solve kar liye), toh specialization (Phase 6) choose karo.



# Phase 5 — Interview Questions & Answers (Hinglish)

---

Real Practice, Lab Setup aur Malware Analysis se related important interview questions, simple Hinglish jawaab ke saath.

## Q1. RE seekhne ke liye practice itni zaroori kyun hai?

**Ans:** RE swimming jaisi hai — kitni bhi theory padho, jab tak khud challenges solve na karo, skill nahi aati. Practice se tumhe pattern pehchanana, tools ka fast use, aur real problems solve karna aata hai. Isliye har hafte crackmes/CTF solve karte rehna zaroori hai.

## Q2. Analysis ke liye Virtual Machine (VM) kyun use karte hain?

**Ans:** VM ek isolated (alag) computer hota hai tumhare PC ke andar. Malware ya unknown files VM mein chalane se tumhara asli system surakshit rehta hai. Agar malware kuch kharaab kare, toh sirf VM affect hota hai, main PC nahi.

## Q3. Snapshot kya hota hai aur kyun useful hai?

**Ans:** Snapshot VM ka "save point" hota hai — us waqt ki poori state. Malware analyze karne ke baad ya kuch bigadne pe, ek click mein us clean snapshot pe wapas aa sakte ho. Isse har baar fresh, clean environment mil jaata hai bina VM dubara banaye.

## Q4. Malware analyze karte waqt network isolation kyun zaroori hai?

**Ans:** Malware often internet se apne C2 (Command & Control) server se baat karta hai, ya network pe phailta hai. Network band/isolate karne se malware baahar nuksan nahi kar sakta, aur na hi tumhara IP/data leak hota hai. Analysis ke liye "host-only" ya no-network rakhte hain.

## Q5. Crackme kya hota hai?

**Ans:** Crackme ek program hota hai jo specially RE practice ke liye banaya jaata hai — legal aur safe. Ismein usually password/serial nikaalna ya koi protection bypass karna hota hai. crackmes.one jaisi sites pe difficulty-wise sorted milte hain, beginners easy se shuru karte hain.

## Q6. CTF kya hota hai?

**Ans:** CTF (Capture The Flag) ek hacking competition hai jismein challenges solve karke chhupe "flags" (secret strings) capture karne hote hain. Reversing category mein binaries ko reverse karke flag nikalte hain. picoCTF beginners ke liye best hai.

---

## Q7. Malware ka static aur dynamic analysis mein kya farak hai?

**Ans: Static malware analysis** malware ko chalaye bina hoti hai — strings, imports, PE structure dekhkar (safe). **Dynamic malware analysis** malware ko VM mein chala ke uska behaviour dekhna hai — kaunse files banata, registry/network kaise use karta. Dono milaake poori picture milti hai.

---

## Q8. IOC (Indicators of Compromise) kya hote hain?

**Ans:** IOCs wo nishaan hain jinse pata chalta hai ki system malware se infected hai — jaise malware ki file hashes, wo kaunse IP/domain se baat karta, kaunsi registry keys ya files banata. Malware analysis mein IOCs nikalte hain taaki detection aur defense ban sake.

---

## Q9. C2 (Command and Control) server kya hota hai?

**Ans:** C2 server wo remote server hai jise attacker control karta hai, aur malware usse instructions leta ya chura hua data bhejta hai. Malware analysis mein hum dekhte hain malware kaunse C2 se, kaise baat karta hai — ye important IOC hota hai.

---

## Q10. Writeup likhna kyun important hai?

**Ans:** Writeup matlab kisi challenge ko kaise solve kiya, wo likhna. Isse concept pakka hota hai (revise ho jaata hai), tum apni thinking clear karte ho, aur ek portfolio banta hai jo jobs/community mein kaam aata hai. Doosron ke writeups padhna bhi nayi tricks sikhata hai.

---

## Q11. Malware analysis se pehle basics solid hona kyun zaroori hai?

**Ans:** Malware often obfuscated, packed, aur anti-analysis tricks se bhara hota hai. Agar assembly, static/dynamic analysis, aur tools ki solid samajh na ho, toh na sirf analysis mushkil hoga, balki galti se malware execute karke apne system ko khatre mein daal sakte ho. Isliye pehle crackmes/CTF pe practice karke basics pakke karte hain.

---

### **Q12. RE mein consistency intensity se zyada important kyun hai?**

**Ans:** RE ek skill hai jo dheere-dheere, regular practice se banti hai. Ek din 10 ghante karke fir mahine bhar chhodne se kam faayda hota hai, bajaye rozana thoda-thoda karne ke. Regular practice se patterns dimaag mein baith jaate hain aur skill permanent ho jaati hai.



# Phase 5 — Interview Questions & Answers (English)

---

The same Real Practice, Lab Setup, and Malware Analysis interview questions, answered in clear English.

---

## Q1. Why is practice so important for learning RE?

**Ans:** RE is like swimming — no matter how much theory you read, the skill comes only from solving challenges yourself. Practice teaches pattern recognition, fast tool usage, and real problem-solving. That's why regularly solving crackmes/CTFs is essential.

---

## Q2. Why do we use a Virtual Machine (VM) for analysis?

**Ans:** A VM is an isolated computer running inside your PC. Running malware or unknown files in a VM keeps your real system safe — if the malware causes damage, only the VM is affected, not your main machine.

---

## Q3. What is a snapshot and why is it useful?

**Ans:** A snapshot is a "save point" of the VM — its complete state at a moment. After analyzing malware or if something breaks, you can revert to that clean snapshot in one click. This gives you a fresh, clean environment each time without rebuilding the VM.

---

## Q4. Why is network isolation important when analyzing malware?

**Ans:** Malware often talks to its C2 (Command & Control) server over the internet or spreads across the network. Isolating the network prevents the malware from causing external harm or leaking your IP/data. For analysis we use "host-only" or no network.

---

## Q5. What is a crackme?

**Ans:** A crackme is a program made specifically for RE practice — legal and safe. It typically requires recovering a password/serial or bypassing some protection. Sites like crackmes.one sort them by difficulty, and beginners start with the easy ones.

---

## Q6. What is a CTF?

**Ans:** A CTF (Capture The Flag) is a hacking competition where you solve challenges to capture hidden "flags" (secret strings). In the reversing category you reverse binaries to extract the flag. picoCTF is best for beginners.

---

## Q7. What is the difference between static and dynamic malware analysis?

**Ans: Static malware analysis** examines malware without running it — looking at strings, imports, and PE structure (safe). **Dynamic malware analysis** runs the malware in a VM to observe its behavior — what files it creates, how it uses the registry/network. Together they give the full picture.

---

## Q8. What are IOCs (Indicators of Compromise)?

**Ans:** IOCs are signs that a system is infected with malware — such as the malware's file hashes, the IPs/domains it contacts, and the registry keys or files it creates. Malware analysis extracts IOCs so defenders can build detection and response.

---

## Q9. What is a C2 (Command and Control) server?

**Ans:** A C2 server is a remote server controlled by the attacker; the malware receives instructions from it or sends stolen data to it. In malware analysis we examine which C2 the malware contacts and how — an important IOC.

---

## Q10. Why is writing writeups important?

**Ans:** A writeup documents how you solved a challenge. It cements the concept (acts as revision), clarifies your thinking, and builds a portfolio useful for jobs and community. Reading others' writeups also teaches new tricks.

---

## Q11. Why must fundamentals be solid before doing malware analysis?

**Ans:** Malware is often obfuscated, packed, and full of anti-analysis tricks. Without a solid grasp of assembly, static/dynamic analysis, and tools, analysis is hard — and you might accidentally execute malware and endanger your system. So we build fundamentals first via crackmes/CTFs.

---

**Q12. Why is consistency more important than intensity in RE?**

**Ans:** RE is a skill built gradually through regular practice. Doing 10 hours one day then nothing for a month helps less than practicing a little every day. Regular practice ingrains patterns in your mind and makes the skill permanent.

## PHASE 6

# Specialization

Apna raasta chuno — malware, exploits, mobile, crypto, firmware

## Phase 6 — Specialization (Apna Raasta Chuno)

**Goal:** Ab tak tum RE ki basics jaante ho. Ab decide karo kis direction mein deep jaana hai. RE bahut bada field hai — sab kuch expert nahi ban sakte, ek-do cheezon mein master bano. Time: **Long term.**

### 6.0 — Specialization Kyun?

Ab tak tum "generalist" ho — thoda sab kuch aata hai. Lekin career aur mastery ke liye ek direction chunni padti hai. Har specialization ka apna tools, techniques, aur job market hai.

Pehle 1-2 areas explore karo (jo interesting lage), phir usme deep jao. Zaroori nahi ek hi mein rehna, but ek time pe ek pe focus.

### 6.1 — Specialization Options

#### 1. 🦟 Malware Analyst

Malware (virus, ransomware, trojans) ko analyze karna. - **Kya karte hain:** Malware kaise kaam karta, kaise phailta, kya chura raha — samajhna. Detection signatures banana. - **Skills:** Static + dynamic analysis, unpacking, Windows internals, network analysis. - **Career:** SOC analyst, threat researcher, incident responder. **Bahut demand.** - **Tools:** Ghidra, x64dbg, Cuckoo Sandbox, Wireshark, YARA.

## 2. 🦋 Vulnerability Researcher / Exploit Developer

Software mein bugs (kamzoriyan) dhoondna aur exploits banana. - **Kya karte hain:** Buffer overflows, use-after-free jaise bugs dhoondna; unhe exploit karna. - **Skills:** Deep assembly, memory corruption, fuzzing, exploit dev (ROP, shellcode). - **Career:** Bug bounty, red team, security research. **High paying.** - **Tools:** Fuzzers (AFL), pwntools, GDB, IDA.

## 3. 📱 Mobile RE (Android / iOS)

Mobile apps ko reverse karna. (Ye Phase 7 mein detail mein — poora plan hai.) - **Kya karte hain:** APK/IPA analyze, app security testing, protection bypass. - **Skills:** Smali (Android), ARM assembly, Frida, mobile frameworks. - **Career:** Mobile app pentester, security researcher.

## 4. 🎮 Game Hacking / Anti-Cheat RE

Games ke internals samajhna (learning ke liye). - **Kya karte hain:** Memory editing, cheats banana/samajhna, anti-cheat research. - **Skills:** Memory analysis, DirectX/OpenGL hooking, C++. - **Note:** Sirf learning/research ke liye — online games mein cheating ban/legal issues laa sakti hai.

## 5. 🗝️ Cryptography RE

Encryption algorithms ko samajhna/todna. - **Kya karte hain:** Custom crypto pehchanna, weak implementations dhoondna. - **Skills:** Math, crypto algorithms (AES, RSA), pattern recognition. - **Career:** Crypto researcher, security auditor.

## 6. ⚡ Firmware / IoT / Embedded RE

Hardware devices ka software (firmware) reverse karna. - **Kya karte hain:** Router, camera, smart devices ka firmware analyze. - **Skills:** ARM/MIPS assembly, firmware extraction, hardware interfaces (UART, JTAG). - **Career:** IoT security researcher, hardware hacker. - **Tools:** binwalk, Ghidra, hardware tools.

## 6.2 — Kaunsa Choose Karein? (Decision Helper)

| Agar tumhe pasand hai...     | Toh ye try karo   |
|------------------------------|-------------------|
| Virus/hacking news, defense  | 🦋 Malware Analyst |
| Bugs dhoondna, puzzles todna | 🦋 Vuln Research   |
| Mobile apps, Android/iOS     | 📱 Mobile RE       |
| Games, memory tricks         | 🎮 Game Hacking    |
| Math, encryption             | 🗝️ Crypto RE      |
| Hardware, gadgets            | ⚡ Firmware/IoT    |

Job market ke hisaab se: **Malware Analysis** aur **Vulnerability Research** ki sabse zyada demand aur pay hai. But jo tumhe maza aaye wahi best hai — kyunki usme tum consistent rahoge.

## 6.3 — Advanced Topics (Jab Ready Ho)

Chahe koi bhi specialization chuno, ye advanced skills sabme kaam aati hain:

### 1. Packers & Unpacking

- Packed binaries (UPX, custom packers) ko unpack karna.
- Manual unpacking — OEP (Original Entry Point) dhoondna, memory dump lena.

### 2. Obfuscation & Deobfuscation

- Code ko jaan-boojh ke complex banana (obfuscation) aur use simplify karna.
- Control flow flattening, junk code, string encryption todna.

### 3. Exploit Development

- Buffer overflow → shellcode → ROP chains.
- Modern protections (ASLR, DEP, stack canaries) aur unhe bypass karna.

### 4. Automation & Scripting (★ Bahut important)

- **Python** seekho — RE mein automation ke liye must.
- **Ghidra scripting** — apne analysis tasks automate karna.
- **angr** — symbolic execution framework (automatic tarike se inputs solve karna).
- **Frida** — dynamic instrumentation (runtime hooking).

### 5. Kernel-level RE

- OS ke andar (drivers, rootkits) ka RE — advanced Windows/Linux internals.

## 6.4 — Career Path (RE Jobs)

RE skills se ye roles mil sakte hain: - **Malware Analyst / Reverse Engineer** — security companies, antivirus vendors. - **Vulnerability Researcher** — product security teams, bug bounty (freelance). - **Security Researcher** — R&D, zero-day research. - **Penetration Tester** — with RE as a specialty (mobile/binary). - **Threat Intelligence Analyst** — malware ko track karke reports banana.

## Portfolio banao

- CTF achievements aur writeups.
  - GitHub pe RE tools/scripts.
  - Blog pe malware analysis / crackme writeups.
  - Certifications (optional): GREM (malware), OSCP/OSED (exploit), etc.
- 

## Key Takeaways (Ek Line Mein)

1. RE bada field hai — ek-do areas mein **master** bano, sab mein nahi.
  2. Options: **Malware, Vuln Research, Mobile, Game, Crypto, Firmware.**
  3. Job demand + pay: **Malware Analysis** aur **Vuln Research** top pe.
  4. Jo tumhe **maza** aaye wahi best — consistency wahin se aayegi.
  5. Advanced skills sabme kaam: **unpacking, deobfuscation, exploit dev, scripting.**
  6. **Python + Ghidra scripting + Frida + angr** — automation ke liye seekho.
  7. **Portfolio** (writeups, CTF, GitHub, blog) career ke liye zaroori.
- 

✅ **Phase 6 Ongoing:** Specialization ek journey hai, ek din ka kaam nahi. Ek area chuno, usme deep jao, projects banao. Aur agla logical step: **Mobile RE (Phase 7)** — poora detailed plan next folder mein. 🚀

# Phase 6 — Interview Questions & Answers (Hinglish)

---

Specialization aur advanced topics se related important interview questions, simple Hinglish jawaab ke saath.

---

## Q1. Reverse Engineering mein specialization kyun zaroori hai?

**Ans:** RE bahut bada field hai — malware, exploits, mobile, crypto, firmware sab alag areas hain, har ek ke apne tools aur techniques hain. Sab mein expert banna mushkil hai, isliye 1-2 areas chunkar unme deep jaana chahiye. Isse mastery aati hai aur career focused hota hai.

---

## Q2. Malware Analyst kya karta hai?

**Ans:** Malware analyst virus, ransomware, trojans jaise malicious software ko analyze karta hai — samajhta hai wo kaise kaam karta, kaise phailta, kya chura raha hai. Fir detection signatures (jaise YARA rules) aur IOCs banata hai taaki systems ko bachaaya ja sake. Ye role security companies mein bahut demand mein hai.

---

## Q3. Vulnerability researcher aur exploit developer kya karte hain?

**Ans:** Ye software mein bugs (kamzoriyan) dhoondte hain — jaise buffer overflow, use-after-free — aur unhe exploit karke dikhate hain ki wo kitne khatarnaak hain. Isse companies bugs fix kar paati hain. Ismein deep assembly, memory corruption, aur exploit development (ROP, shellcode) ki samajh chahiye. Bug bounty aur red team mein ye skills kaam aati hain.

---

## Q4. Packer kya hota hai aur unpacking kya hoti hai?

**Ans:** Packer ek tool hai jo program ke code ko compress ya encrypt kar deta hai (jaise UPX), taaki analysis mushkil ho aur size chhota ho. Unpacking matlab us packed code ko wapas asli form mein laana — usually program ko chala ke, jab wo khud ko memory mein unpack kar leta hai, tab OEP (Original Entry Point) pe memory dump le lete hain.

---



## Q5. Obfuscation kya hoti hai?

**Ans:** Obfuscation matlab code ko jaan-boojh ke complex/confusing banana taaki reverse karna mushkil ho — bina behaviour badle. Techniques: junk (bekaar) code daalna, control flow flatten karna, strings encrypt karna, naam ulte-pulte karna. Deobfuscation ismein wo confusion hataake asli logic nikaalna hai.

---

## Q6. Exploit development mein ASLR, DEP, stack canary kya hote hain?

**Ans:** Ye modern security protections hain jo exploits ko rokhte hain: - **ASLR** — memory addresses ko har baar random kar deta hai, taaki attacker ko address pata na chale. - **DEP/NX** — data wale memory ko execute hone se rokta hai (shellcode na chale). - **Stack canary** — stack pe ek secret value rakhta hai; overflow se wo change ho jaaye toh program detect kar leta hai.

Exploit developers inhe bypass karne ke techniques (jaise ROP) seekhte hain.

---

## Q7. RE mein scripting/automation kyun important hai?

**Ans:** Bahut sa RE kaam repetitive hota hai (patterns dhoondna, decrypt karna, inputs solve karna). Python jaise scripting se ye automate ho jaata hai. Ghidra scripting se analysis tasks, angr se automatic input solving (symbolic execution), aur Frida se runtime hooking automate hoti hai. Isse time bachta hai aur bade programs handle hote hain.

---

## Q8. angr framework kya hai?

**Ans:** angr ek Python-based binary analysis framework hai jo "symbolic execution" karta hai — matlab program ko symbolically chalaakar automatically wo input dhoond leta hai jo kisi specific path (jaise "access granted") tak le jaaye. RE mein ise password/key automatically solve karne ke liye use karte hain.

---

## Q9. Firmware RE aam software RE se kaise alag hai?

**Ans:** Firmware embedded devices (routers, cameras, IoT) ka software hota hai. Ye often ARM/MIPS architecture pe hota hai (x86 nahi), aur pehle device se firmware extract karna padta hai (binwalk jaise tools se). Kabhi hardware interfaces (UART, JTAG) se seedha chip se code nikalna padta hai. Baaki RE concepts same rehte hain.

---

### **Q10. YARA rules kya hote hain?**

**Ans:** YARA rules ek tarah ke "patterns" hain jinse malware ko identify/detect kiya jaata hai. Rule mein hum specific strings, bytes, ya conditions likhte hain jo kisi malware family mein milte hain. Malware analysts ye rules banate hain taaki antivirus/scanners similar malware pakad sakein.

---

### **Q11. RE mein career ke kaunse options hain?**

**Ans:** Malware Analyst, Reverse Engineer, Vulnerability Researcher, Security Researcher, Penetration Tester (binary/mobile specialty), aur Threat Intelligence Analyst. Ye roles security companies, antivirus vendors, product security teams, aur bug bounty (freelance) mein hote hain.

---

### **Q12. Specialization choose karte waqt kaunsi cheez sabse zyada matter karti hai?**

**Ans:** Job demand aur pay important hain (Malware Analysis aur Vuln Research top pe hain), lekin sabse zyada matter karta hai ki tumhe kis cheez mein maza aata hai. Kyunki RE lambi mehnat maangti hai, aur consistency wahin aayegi jahan interest hoga. Interest + demand ka combo best hai.

# Phase 6 — Interview Questions & Answers (English)

---

The same Specialization and advanced-topics interview questions, answered in clear English.

---

## Q1. Why is specialization important in Reverse Engineering?

**Ans:** RE is a very broad field — malware, exploits, mobile, crypto, and firmware are all distinct areas with their own tools and techniques. Becoming expert in everything is impractical, so you pick 1-2 areas and go deep. This builds mastery and gives your career focus.

---

## Q2. What does a Malware Analyst do?

**Ans:** A malware analyst examines malicious software like viruses, ransomware, and trojans — understanding how it works, spreads, and what it steals. They then create detection signatures (e.g., YARA rules) and IOCs to protect systems. The role is in high demand at security companies.

---

## Q3. What do vulnerability researchers and exploit developers do?

**Ans:** They find bugs in software — such as buffer overflows and use-after-free — and build exploits to demonstrate their severity, so companies can fix them. It requires deep assembly, understanding of memory corruption, and exploit development (ROP, shellcode). These skills are used in bug bounties and red teaming.

---

## Q4. What is a packer and what is unpacking?

**Ans:** A packer is a tool that compresses or encrypts a program's code (e.g., UPX) to make analysis harder and reduce size. Unpacking means restoring the original code — usually by running the program until it unpacks itself in memory, then dumping the memory at the OEP (Original Entry Point).

---

## Q5. What is obfuscation?

**Ans:** Obfuscation is deliberately making code complex/confusing to hinder reversing, without changing its behavior. Techniques include inserting junk code, flattening control flow, encrypting strings, and mangling names. Deobfuscation removes that confusion to recover the real logic.

---

## Q6. In exploit development, what are ASLR, DEP, and stack canaries?

**Ans:** These are modern protections that hinder exploits: - **ASLR** — randomizes memory addresses each run, so attackers can't predict them. - **DEP/NX** — marks data memory as non-executable, preventing shellcode from running. - **Stack canary** — places a secret value on the stack; if an overflow changes it, the program detects the tampering.

Exploit developers learn techniques (like ROP) to bypass these.

---

## Q7. Why is scripting/automation important in RE?

**Ans:** Much RE work is repetitive (finding patterns, decrypting, solving inputs). Scripting with Python automates it. Ghidra scripting automates analysis tasks, angr automates input solving (symbolic execution), and Frida automates runtime hooking. This saves time and scales to large programs.

---

## Q8. What is the angr framework?

**Ans:** angr is a Python-based binary analysis framework that performs "symbolic execution" — running a program symbolically to automatically find the input that reaches a specific path (like "access granted"). In RE it's used to automatically solve passwords/keys.

---

## Q9. How is firmware RE different from ordinary software RE?

**Ans:** Firmware is the software of embedded devices (routers, cameras, IoT). It's often on ARM/MIPS architectures (not x86), and you must first extract the firmware from the device (with tools like binwalk). Sometimes you extract code directly from the chip via hardware interfaces (UART, JTAG). The core RE concepts remain the same.

---

## Q10. What are YARA rules?

**Ans:** YARA rules are patterns used to identify/detect malware. In a rule you specify strings, bytes, or conditions characteristic of a malware family. Analysts write these rules so antivirus/scanners can catch similar malware.

---

## Q11. What career options exist in RE?

**Ans:** Malware Analyst, Reverse Engineer, Vulnerability Researcher, Security Researcher, Penetration Tester (with a binary/mobile specialty), and Threat Intelligence Analyst. These roles exist at security companies, antivirus vendors, product security teams, and in bug bounty (freelance).

---

## **Q12. What matters most when choosing a specialization?**

**Ans:** Job demand and pay matter (Malware Analysis and Vulnerability Research are top), but what matters most is what you genuinely enjoy. RE demands sustained effort, and consistency comes from interest. The best choice combines interest with demand.

## PHASE 7

# Mobile RE

Android (APK, smali) aur iOS (IPA) apps reverse karna

## Phase 7 — Mobile Reverse Engineering (Android + iOS)

**Goal:** Mobile apps (Android APK, iOS IPA) ko reverse karna seekhna. Ye Phase 0-6 ke concepts (assembly, static/dynamic, debugging) use karta hai, bas platform alag hai. Time: **6-8 hafte** (dono platforms). Base strong hoga toh aasan lagega.

### 7.0 — Mobile RE Kyun Alag Hai?

Desktop `.exe` aur mobile apps mein bada farak hai: - **Android apps** Java/Kotlin mein banti hain aur ek special "bytecode" (DEX) mein chalti hain — jo reverse karna **aasan** hai (bytecode → aasani se wapas Java jaisa). - **iOS apps** Swift/Objective-C mein banti hain aur seedha **native ARM machine code** mein — jo desktop RE jaisa **mushkil** hai.

Isliye tools aur approach dono alag hain. Ye phase do parts mein hai: - **Part A — Android RE** (pehle karo — free, aasan) - **Part B — iOS RE** (baad mein — Mac + jailbroken device chahiye)

## PART A — ANDROID REVERSE ENGINEERING

 **Time: 4-5 hafte**

Android se shuru karo — free hai, tools easily milte hain, normal Windows/Linux PC pe ho jaata hai.

## 7A.1 — Android Basics (Neev)

### APK file kya hai?

**APK = Android Package.** Ye asal mein ek **ZIP file** hai! `app.apk` ko `app.zip` rename karke khol lo — andar sab files dikh jaayengi.

### APK ke andar kya hota hai

| File/Folder                      | Kya Hai                                                              |
|----------------------------------|----------------------------------------------------------------------|
| <code>AndroidManifest.xml</code> | App ka "identity card" — permissions, components, entry point        |
| <code>classes.dex</code>         | ★ Asli code (Dalvik bytecode) — yahin logic chhupa hai               |
| <code>resources.arsc</code>      | Compiled resources (strings, layouts)                                |
| <code>res/</code>                | Images, layouts, icons                                               |
| <code>lib/</code>                | Native code ( <code>.so</code> files — C/C++ compiled, ARM assembly) |
| <code>assets/</code>             | Extra files (fonts, DBs, config)                                     |
| <code>META-INF/</code>           | Signature (app kis ne sign kiya)                                     |

### App kaise banti hai (samajhna zaroori)

```
Java/Kotlin Code
  ↓ (compile)
.class (Java bytecode)
  ↓ (dx / d8 tool)
.dex (Dalvik bytecode)  <- RE mein hum ISKO wapas padhte hain
  ↓ (package + sign)
APK
```

### DVM vs ART

Android apps ek Virtual Machine pe chalti hain — pehle **Dalvik VM (DVM)**, ab **ART (Android Runtime)**. Isliye code seedha "machine code" nahi, balki **bytecode** hota hai — jo desktop se reverse karna aasan hai!

**Key point:** DEX bytecode high-level info (class names, method names) rakhta hai, isliye reverse karke wapas Java jaisa code aasani se milta hai.

## 7A.2 — Smali Language (Android Ki "Assembly")

Ye Android RE ka **dil** hai. Jaise desktop mein Assembly thi, waise Android mein **Smali** hai.

### Smali kya hai?

Smali, Dalvik bytecode ka human-readable version hai. `.dex` ko todoge toh smali dikhega. Ye Java se low-level hai but desktop assembly se aasan (registers aur methods clearly dikhte hain).

### Example — Ek simple smali

```
.method public checkPassword(Ljava/lang/String;)Z
    .locals 2
    const-string v0, "admin123"          # v0 register mein "admin123" daalo
    invoke-virtual {p1, v0}, Ljava/lang/String;->equals(Ljava/lang/Object;)Z
    move-result v1                        # equals() ka result v1 mein
    return v1                             # true/false return
.end method
```

**Line by line:** - `.method public checkPassword(...)Z` — method ka naam, `Z` matlab boolean (true/false) return karega. - `const-string v0, "admin123"` — v0 register mein string "admin123" daali. - `invoke-virtual {p1, v0}, ...equals` — `p1` (user input) pe `.equals(v0)` call kiya — matlab input ko "admin123" se compare. - `move-result v1` — comparison ka result v1 mein. - `return v1` — result wapas.

**Password mil gaya:** `admin123` — smali padhna aa gaya toh aadha kaam ho gaya!

### Smali seekhne ke key points

- **Registers:** `v0, v1...` (local) aur `p0, p1...` (parameters; `p0` = `this` object).
- **Common opcodes:** `const-string`, `invoke-virtual`, `invoke-static`, `if-eq`, `if-ne`, `move-result`, `return`, `goto`.
- **Types:** `Z` =boolean, `I` =int, `V` =void, `Ljava/lang/String;` =String object.



## 7A.3 — Android RE Tools

| Tool                             | Kaam                                                             | Level |
|----------------------------------|------------------------------------------------------------------|-------|
| <b>jadx / jadx-gui</b> ★         | APK ko seedha Java code mein dikhaata hai (best beginner tool)   | Free  |
| <b>apktool</b> ★                 | APK ko smali mein todta hai aur wapas build karta hai (patching) | Free  |
| <b>dex2jar</b>                   | <code>.dex</code> ko <code>.jar</code> mein badalta hai          | Free  |
| <b>JD-GUI / Bytecode Viewer</b>  | Java code dekhna                                                 | Free  |
| <b>Ghidra / IDA</b>              | <code>lib/*.so</code> (native C/C++) reverse karna               | Free  |
| <b>Android Studio + Emulator</b> | App chalana/debug karna                                          | Free  |
| <b>Frida</b> ★★                  | Runtime hooking (dynamic) — sabse powerful                       | Free  |

**Beginner recommendation:** **jadx-gui** se shuru karo. APK drag-drop karo, turant Java-jaisa code dikh jaayega.

## 7A.4 — Android Static Analysis

Bina app chalaye samajhna:

1. **APK ko jadx-gui mein kholo** → Java code padho.
2. **AndroidManifest.xml padho** → kaunsi Activity pehle chalti hai (entry point, usually `MainActivity`), kya permissions maangi.
3. **Strings dhoondo** → URLs, API keys, passwords aksar hardcoded milte hain.
4. **Interesting functions** → login, password check, encryption wale functions dhoondo.
5. **Native `.so` files** → agar logic native code mein hai toh Ghidra mein kholo (yahan Phase 1 ki ARM assembly kaam aayegi!).

### Real example flow

```
Ek "Login" app mila
→ jadx-gui mein khola
→ MainActivity dhoondi
→ checkLogin() function mila
→ dekha: if(password.equals("Sup3rS3cr3t"))
→ Password mil gaya bina app chalaye! 🎉
```

## 7A.5 — Android Dynamic Analysis

App ko **chale hue** dekhna: - **Emulator setup** — Android Studio ka emulator (safe, computer pe hi Android chalega). Rooted emulator best hai. - **Logcat** — app ke logs dekhna (developers debug info chhod dete hain). - **Frida** ★★ — Sabse powerful tool. Chalti hui app mein "hook" laga ke functions ko rok/badal sakte ho, runtime values dekh sakte ho. - Example: `equals()` function hook karke dekho asli value kya compare ho rahi. - **Objection** — Frida ke upar bana, aasan commands (SSL pinning bypass, root detection bypass). - **Burp Suite / mitmproxy** — app ka network traffic dekhna (kaunse server se baat kar rahi).

### Frida hooking example (concept)

```
// login function ko hook karke uske arguments print karna
Java.perform(function() {
    var Login = Java.use("com.app.LoginActivity");
    Login.checkPassword.implementation = function(pass) {
        console.log("Password argument: " + pass); // asli input print
        return true; // hamesha true return - bypass!
    };
});
```

**Samjho:** - `Java.use(...)` — target class pakdi. - `.implementation = function(...)` — us method ko replace kiya. - `console.log(pass)` — jo bhi password aaya, print kar diya. - `return true` — function ko hamesha "sahi" bana diya (login bypass).

## 7A.6 — APK Patching (Modify & Rebuild)

App ka behaviour badalna:

```
apktool d app.apk          # 1. APK ko smali mein todo (decompile)
# 2. smali file edit karo (jaise "if-eqz" ko badlo ya return value change karo)
apktool b app_folder -o new.apk # 3. wapas APK banao (build)
# 4. APK ko sign karo (warna install nahi hoga)
apksigner sign --ks my.keystore new.apk
# 5. install & test (emulator/phone pe)
adb install new.apk
```

Example: Ek "premium locked" feature ko smali mein `return true` bana ke unlock karna (learning ke liye). Ya login check ko hamesha pass karwana.

⚠️ Sirf apni ya practice apps pe. Kisi ki paid app ki protection todna illegal hai.

## 7A.7 — Android Advanced Topics

- **Obfuscation** — ProGuard/R8/DexGuard se naam ulte (`a.a.a()`) — inhe trace karna.
- **Root detection bypass** — apps jo rooted device pe chalne se mana karti hain.
- **SSL Pinning bypass** — network traffic dekhne ke liye (Objection/Frida se).
- **Anti-tampering bypass** — apps jo modify hona detect karti hain.
- **Native code RE** — `.so` files (ARM assembly) — desktop RE skills seedha kaam aati hain.
- **Frida scripting** — apne custom hooks likhna.

✅ **Android Part Complete Jab:** Tum ek APK ko jadx mein khol ke logic samajh sako, smali padh sako, Frida se ek function hook kar sako, aur ek simple patch karke APK rebuild kar sako.



## PART B — iOS REVERSE ENGINEERING

🕒 **Time: 3-4 hafte**

⚠️ **Note:** iOS RE thoda mushkil aur mehanga hai. Ideally chahiye: ek **Mac** (kuch tools sirf Mac pe) aur ek **jailbroken iPhone** (dynamic analysis ke liye). Bina inke bhi static analysis ho sakta hai, but limited. Isliye Android pehle solid karo.

iOS apps **Swift/Objective-C** mein banti hain aur seedha **ARM machine code** (native) mein compile hoti hain — Android ke bytecode jaisa aasan nahi. Yahan desktop RE skills (assembly, Ghidra) zyada kaam aati hain.

## 7B.1 — iOS App Basics

### IPA file kya hai?

Android ke APK ki tarah, iOS ka app package. Ye bhi ek **ZIP** hai! Andar `Payload/AppName.app/` folder hota hai.

## IPA ke andar kya hota hai

| File                              | Kya Hai                                              |
|-----------------------------------|------------------------------------------------------|
| <code>Payload/AppName.app/</code> | Asli app folder                                      |
| <code>AppName</code> (binary)     | ★ Main executable — Mach-O format (ARM machine code) |
| <code>Info.plist</code>           | App ki config/identity (Android ke Manifest jaisa)   |
| <code>Frameworks/</code>          | Extra libraries                                      |
| <code>_CodeSignature/</code>      | Apple ka digital signature                           |
| Assets ( <code>.car</code> )      | Images, resources                                    |

## Mach-O format

iOS/Mac binaries ka format (jaise Windows mein PE, Linux mein ELF). Ghidra/IDA ise samajhte hain.

## Encryption problem ( ★ important)

App Store se aane wali apps **encrypted** hoti hain (FairPlay DRM). Reverse karne ke liye pehle **decrypt** karni padti hai — jailbroken device pe, jab app memory mein decrypt ho jaati hai tab use dump karke (jaise `frida-ios-dump` se).

## 7B.2 — Objective-C & Swift Samjho

- **Objective-C** — purani but common. Ismein method calls ek special function `objc_msgSend` ke through hote hain — ye RE mein bahut dikhega.
- **Swift** — modern, but reverse karna thoda mushkil (name mangling — naam encode ho jaate hain).
- Basic samajh honi chahiye ki inka code kaisa dikhta hai — classes, methods, message passing.

## 7B.3 — iOS RE Tools

| Tool                       | Kaam                                          | Note         |
|----------------------------|-----------------------------------------------|--------------|
| <b>Ghidra / IDA Pro</b> ★  | Mach-O binary disassemble/decompile           | Free/Paid    |
| <b>Hopper Disassembler</b> | iOS/Mac ke liye popular, achha UI             | Paid (trial) |
| <b>class-dump</b>          | Objective-C classes/methods ki list nikaalna  | Free         |
| <b>Frida</b> ★             | Dynamic analysis, hooking (jailbroken device) | Free         |
| <b>Objection</b>           | Frida-based, aasan commands                   | Free         |
| <b>frida-ios-dump</b>      | Encrypted app ko decrypt/dump karna           | Free         |
| <b>otool / nm</b>          | Binary info, symbols dekhna (Mac tools)       | Free         |

## 7B.4 — iOS Static Analysis

1. **IPA ko unzip karo** → binary aur `Info.plist` nikaalo.
2. **Info.plist padho** → permissions, URL schemes, config.
3. **App decrypt karo** (agar App Store se) → `frida-ios-dump` se.
4. **class-dump** → saari classes aur methods ki list (kya-kya functions hain).
5. **Ghidra/Hopper mein binary kholo** → interesting functions (login, crypto) dhoondo, ARM assembly padho.
6. **Strings** → hardcoded secrets, URLs.

## 7B.5 — iOS Dynamic Analysis

Iske liye **jailbroken iPhone** chahiye (ya kuch cases mein simulator, but limited). - **Frida hooking** — chalti hui app ke functions rok/badal ke dekhna (Android jaisa hi, but Objective-C/Swift methods). - **Objection** — SSL pinning bypass, jailbreak detection bypass, method hooking aasan commands se. - **Debugserver / LLDB** — Apple ka debugger, breakpoints lagana. - **Network analysis** — Burp Suite se traffic dekhna. - **Cycript / Frida REPL** — live app mein objects explore karna.

## 7B.6 — iOS Advanced Topics

- **FairPlay decryption** deeply samajhna.
- **Anti-jailbreak detection bypass** — apps jo jailbroken device pe chalne se mana karti hain.
- **SSL pinning bypass** — network dekhne ke liye.
- **Swift-specific RE** — name demangling, metadata.
- **Kernel-level / system RE** — bahut advanced.

✅ **iOS Part Complete Jab:** Tum ek IPA ko unzip/analyze kar sako, class-dump se structure samajh sako, Ghidra/Hopper mein binary explore kar sako, aur (jailbroken device ho toh) Frida se hook laga sako.

## 🔑 Android vs iOS — Quick Comparison

| Cheez      | Android                       | iOS                               |
|------------|-------------------------------|-----------------------------------|
| Package    | <code>.apk</code> (ZIP)       | <code>.ipa</code> (ZIP)           |
| Code type  | Bytecode (Dalvik/DEX) — aasan | Native ARM machine code — mushkil |
| Language   | Java / Kotlin                 | Swift / Objective-C               |
| "Assembly" | Smali                         | ARM Assembly                      |
| Main tool  | jadx, apktool, Frida          | Ghidra/Hopper, Frida              |
| Encryption | Usually nahi                  | Haan (FairPlay DRM)               |
| Setup cost | Free (koi PC)                 | Mehnga (Mac + jailbroken iPhone)  |
| Difficulty | 🟢 Aasan (shuru yahin se)      | 🔴 Mushkil (baad mein)             |

**Rule of thumb:** Android pehle poora seekho (aasan + free), phir iOS. **Frida dono mein common hai** — isko achhe se seekhna dono jagah kaam aayega.

## 📱 Mobile RE — Practice & Resources

- 🌐 **OWASP MASTG (Mobile App Security Testing Guide)** — mobile RE/security ki bible, free.
- 🌐 **MASVS** — mobile security standards.
- 🧪 **DIVA / InsecureBankv2 / OWASP UnCrackable** — jaan-boojh ke kamzor banaayi gayi practice apps (Android + iOS dono).

- 📱 Apni khud ki banaayi hui simple app reverse karke shuru karo (safe + legal).
  - 📺 YouTube: LiveOverflow (mobile), 8ksec, TheGrayArea.
- 

## 🎯 Key Takeaways (Ek Line Mein)

1. **APK aur IPA dono ZIP files hain** — rename karke khol sakte ho.
  2. Android code = **bytecode (DEX)** — aasan; iOS code = **native ARM** — mushkil.
  3. **Smali** = Android ki assembly; padhna aa gaya toh aadha kaam.
  4. Android tools: **jadx** (Java dekho), **apktool** (patch), **Frida** (hook).
  5. iOS apps **encrypted** (FairPlay) — pehle decrypt/dump karni padti (frida-ios-dump).
  6. iOS tools: **Ghidra/Hopper**, **class-dump**, **Frida**, **Objection**.
  7. **Frida dono platforms mein** sabse powerful — zaroor seekho.
  8. **Android pehle** (free), phir iOS (Mac + jailbroken device).
- 

✅ **Phase 7 Complete Jab:** Tum Android APK aur (resources ho toh) iOS IPA dono ko static + dynamic dono tarike se analyze kar sako, aur ek practice app ka logic/secret nikaal sako. Congratulations — ab tum ek Mobile RE bhi ho! 🎉 🚀

# Phase 7 — Interview Questions & Answers (Hinglish)

---

Mobile RE (Android + iOS) se related important interview questions, simple Hinglish jawaab ke saath.

---

## Q1. APK file kya hoti hai?

**Ans:** APK (Android Package) ek Android app ka installation package hai. Ye asal mein ek **ZIP file** hoti hai — rename karke khol sakte ho. Andar `AndroidManifest.xml`, `classes.dex` (asli code), resources, `lib/` (native code), aur signature hoti hai.

---

## Q2. `classes.dex` file mein kya hota hai?

**Ans:** `classes.dex` mein app ka asli code hota hai — Dalvik bytecode ke form mein. Ye wahi code hai jo Android ki virtual machine (ART/Dalvik) chalati hai. RE mein isi ko reverse karke wapas Java/smali mein padhte hain.

---

## Q3. Smali kya hai?

**Ans:** Smali, Dalvik bytecode ka human-readable version hai — Android ki "assembly language". Jab `.dex` ko decompile karte hain toh smali milta hai. Ye Java se low-level hai but desktop assembly se aasan, kyunki ismein registers ( `v0`, `p1` ) aur method names clearly dikhte hain.

---

## Q4. DVM/ART kya hai aur Android code reverse karna aasan kyun hota hai?

**Ans:** Android apps ek virtual machine pe chalti hain — pehle Dalvik VM (DVM), ab ART (Android Runtime). Isliye app ka code seedha machine code nahi, balki **bytecode** hota hai jo high-level info (class/method names) rakhta hai. Isliye reverse karke wapas Java-jaisa readable code aasani se mil jaata hai — desktop native code se kaafi aasan.

---

## Q5. jadx aur apktool mein kya farak hai?

**Ans:** **jadx** APK ko seedha readable Java code mein dikhata hai — code samajhne (static analysis) ke liye best. **apktool** APK ko smali mein todta hai aur usse wapas APK build bhi kar sakta hai — matlab patching (code badalkar rebuild) ke liye use hota hai.



---

## Q6. AndroidManifest.xml kyun important hai?

**Ans:** Ye app ka "identity card" hai. Ismein app ke saare components (Activities, Services), permissions (kya-kya access maangti hai), aur entry point (kaunsi Activity pehle chalti hai, usually MainActivity) hote hain. RE shuru karte waqt yahan se pata chalta hai app kahan se shuru hoti hai aur kya karti hai.

---

## Q7. Frida kya hai aur mobile RE mein kyun important hai?

**Ans:** Frida ek dynamic instrumentation tool hai jo chalti hui app mein functions ko "hook" karne deta hai — matlab unhe rok sakte ho, arguments/return values dekh/badal sakte ho, ya poora replace kar sakte ho. Ye Android aur iOS dono mein kaam karta hai, isliye mobile RE ka sabse powerful aur zaroori tool hai (jaise login bypass, SSL pinning bypass).

---

## Q8. APK patching ka process kya hai?

**Ans:** Steps: (1) `apktool d app.apk` se APK ko smali mein decompile karo, (2) smali code edit karo (jaise koi check ka result badal do), (3) `apktool b` se wapas APK build karo, (4) APK ko sign karo (`apksigner`), warna install nahi hoga, (5) `adb install` se emulator/phone pe install karke test karo.

---

## Q9. IPA file kya hoti hai aur APK se kaise alag hai?

**Ans:** IPA iOS app ka package hai (APK ka iOS version), aur ye bhi ek ZIP file hai jisme `Payload/AppName.app/` folder hota hai. Bada farak: IPA ke andar binary **native ARM machine code** (Mach-O format) mein hoti hai — Android ke aasan bytecode se mushkil. Aur App Store apps **encrypted** (FairPlay) hoti hain.

---

## Q10. iOS apps reverse karna Android se mushkil kyun hai?

**Ans:** Do wajah: (1) iOS apps native ARM machine code mein compile hoti hain (Android ki tarah high-level bytecode nahi), isliye readable code wapas nahi milta — assembly padhni padti hai. (2) App Store apps FairPlay DRM se encrypted hoti hain, jinhe pehle jailbroken device pe decrypt/dump karna padta hai. Plus setup mehnga hai (Mac + jailbroken iPhone).

---

## Q11. FairPlay encryption kya hai aur ise kaise handle karte hain?

**Ans:** FairPlay Apple ka DRM hai jo App Store apps ki binary ko encrypt kar deta hai, taaki koi seedha reverse na kar sake. Handle karne ke liye jailbroken device pe app ko chalate hain — jab app memory

mein khud ko decrypt kar leti hai, tab `frida-ios-dump` jaise tool se decrypted binary memory se dump kar lete hain, jise phir Ghidra/Hopper mein analyze karte hain.

---

### Q12. class-dump tool kya karta hai?

**Ans:** class-dump iOS/Mac binary se Objective-C ki saari classes, methods, aur properties ki list nikaal deta hai (header files jaise). Isse ek hi jhalak mein pata chal jaata hai ki app mein kaunse functions hain — jaise `checkLogin`, `isPremium` — aur kahan analysis focus karni hai.

---

### Q13. SSL pinning kya hai aur use bypass kyun karte hain?

**Ans:** SSL pinning ek security technique hai jismein app sirf ek specific certificate ko trust karti hai, taaki koi beech mein (proxy) traffic na dekh sake. RE/testing mein hum app ka network traffic dekhna chahte hain (Burp Suite se), isliye SSL pinning ko bypass karte hain (Objection/Frida se), taaki traffic proxy se guzre aur dikh sake.

---

### Q14. Android aur iOS RE mein kya common cheez seekhna best hai?

**Ans: Frida** — kyunki ye dono platforms mein kaam karta hai. Ek baar Frida hooking achhe se seekh li toh Android aur iOS dono ki dynamic analysis (function hooking, bypass, runtime inspection) cover ho jaati hai. Isliye ye mobile RE ka sabse high-value skill hai.

---

### Q15. Mobile app ki native `.so` files kya hoti hain aur unhe kaise reverse karte hain?

**Ans:** `.so` files Android APK ke `lib/` folder mein hoti hain — ye C/C++ mein likha, ARM assembly mein compiled native code hai (jaise performance ya security-sensitive logic). Inhe reverse karne ke liye Ghidra/IDA use karte hain — yahan desktop RE aur ARM assembly ki skills seedha kaam aati hain, kyunki ye bytecode nahi native code hai.

# Phase 7 — Interview Questions & Answers (English)

---

The same Mobile RE (Android + iOS) interview questions, answered in clear English.

---

## Q1. What is an APK file?

**Ans:** An APK (Android Package) is an Android app's installation package. It's actually a **ZIP file** — you can rename and open it. Inside are `AndroidManifest.xml`, `classes.dex` (the actual code), resources, `lib/` (native code), and the signature.

---

## Q2. What is stored in the `classes.dex` file?

**Ans:** `classes.dex` contains the app's actual code as Dalvik bytecode — the code Android's virtual machine (ART/Dalvik) runs. In RE we reverse it back into Java/smali to read it.

---

## Q3. What is Smali?

**Ans:** Smali is the human-readable form of Dalvik bytecode — Android's "assembly language." Decompiling a `.dex` yields smali. It's lower-level than Java but easier than desktop assembly, since it clearly shows registers ( `v0`, `p1` ) and method names.

---

## Q4. What is DVM/ART, and why is Android code easier to reverse?

**Ans:** Android apps run on a virtual machine — formerly Dalvik VM (DVM), now ART (Android Runtime). So the app's code is **bytecode**, not raw machine code, and it retains high-level info (class/method names). This makes it easy to reverse back into readable Java-like code — far easier than desktop native code.

---

## Q5. What is the difference between `jadx` and `apktool`?

**Ans:** `jadx` shows an APK directly as readable Java code — best for understanding code (static analysis). `apktool` breaks an APK into smali and can rebuild it into an APK — used for patching (modifying code and rebuilding).

---

## Q6. Why is AndroidManifest.xml important?

**Ans:** It's the app's "identity card." It lists all components (Activities, Services), permissions (what access it requests), and the entry point (which Activity launches first, usually MainActivity). When starting RE, it tells you where the app begins and what it does.

---

## Q7. What is Frida and why is it important in mobile RE?

**Ans:** Frida is a dynamic instrumentation tool that lets you "hook" functions in a running app — pause them, view/modify arguments and return values, or replace them entirely. It works on **both** Android and iOS, making it the most powerful and essential mobile RE tool (e.g., login bypass, SSL pinning bypass).

---

## Q8. What is the APK patching process?

**Ans:** Steps: (1) `apktool d app.apk` to decompile the APK into smali, (2) edit the smali code (e.g., change a check's result), (3) `apktool b` to rebuild the APK, (4) sign the APK ( `apksigner` ), otherwise it won't install, (5) `adb install` to install and test on an emulator/phone.

---

## Q9. What is an IPA file and how does it differ from an APK?

**Ans:** An IPA is an iOS app package (the iOS counterpart of an APK), and it's also a ZIP file containing a `Payload/AppName.app/` folder. Key difference: the IPA's binary is **native ARM machine code** (Mach-O format) — harder than Android's bytecode. Also, App Store apps are **encrypted** (FairPlay).

---

## Q10. Why is reversing iOS apps harder than Android?

**Ans:** Two reasons: (1) iOS apps compile to native ARM machine code (not high-level bytecode like Android), so you don't get readable code back — you must read assembly. (2) App Store apps are encrypted with FairPlay DRM and must first be decrypted/dumped on a jailbroken device. Plus the setup is expensive (Mac + jailbroken iPhone).

---

## Q11. What is FairPlay encryption and how do you handle it?

**Ans:** FairPlay is Apple's DRM that encrypts App Store app binaries so they can't be directly reversed. To handle it, you run the app on a jailbroken device — once the app decrypts itself in memory, you dump the decrypted binary from memory using a tool like `frida-ios-dump`, then analyze it in Ghidra/Hopper.

---

### Q12. What does the class-dump tool do?

**Ans:** class-dump extracts all Objective-C classes, methods, and properties from an iOS/Mac binary (like header files). At a glance it reveals which functions exist — such as `checkLogin`, `isPremium` — and where to focus your analysis.

---

### Q13. What is SSL pinning and why do you bypass it?

**Ans:** SSL pinning is a security technique where an app trusts only a specific certificate, so no one in the middle (a proxy) can read its traffic. In RE/testing we want to see the app's network traffic (via Burp Suite), so we bypass SSL pinning (using Objection/Frida) to let the traffic pass through the proxy and be visible.

---

### Q14. What common skill is best to learn for both Android and iOS RE?

**Ans: Frida** — because it works on both platforms. Once you learn Frida hooking well, you cover dynamic analysis (function hooking, bypasses, runtime inspection) for both Android and iOS. It's the highest-value skill in mobile RE.

---

### Q15. What are a mobile app's native `.so` files and how do you reverse them?

**Ans:** `.so` files live in an Android APK's `lib/` folder — they're native code written in C/C++ and compiled to ARM assembly (for performance- or security-sensitive logic). You reverse them with Ghidra/IDA — here your desktop RE and ARM assembly skills apply directly, since this is native code, not bytecode.